

An Analysis of Fish Data in the North East Texas and South East Oklahoma

In this file I have combined all of the scripts, functions, and figures that I have found insightful or relevant towards the interpretation of this data.

Table of contents

- **An Analysis of Fish Data in the North East Texas and South East Oklahoma**
 - Importing all of the Necessary Libraries
 - Loading the Files as Pandas Data Frames & Global Vars
 - Bar Plots
 - Total Fish Species Bar Plot
 - Total Fish Genus Bar Plot
 - Total Population and Genus at Each Site
 - Total Population and Genus at Each Site Ordered Bar Plot
 - Site Attribute Bar Plots
 - High t_population Sites
 - Site #29 Bar Plots
 - Site #4 Bar Plots
 - Site #9 Bar Plots
 - Site #43 Bar Plots
 - Low t_population Sites
 - Site #1 Bar Plots
 - Site #18 Bar Plots
 - Site #2 Bar Plots
 - Site #17 Bar Plots
 - Correlation Plots
 - **seth_envData_2025.csv** Correlation Heatmap
 - Ranked Correlation Decay Plot
 - Scatter TrendLine Plotter
 - pH Scatter Plot
 - upstreamCumDA_km2 Scatter Plot
 - pb_Bedrock Scatter Plot
 - pft_macrophytes Scatter Plot
 - Clustering Methods
 - Clustering Sites by envAtt
 - KNN Classifier
 - A Focus on Orange Belly Darters
 - Etheostoma Radiosum Distribution

- Sites Attributes with High Orange Belly Darter Count
- Correlation Decay Plot With Site Attributes
- Specific Attribute Trend Lines
- U-MAP Clustering
- KNN Classifier

Importing all of the Necessary Libraries

As I write this code, I will import **ALL** of the libraries I use in this cell

```
In [48]: # ===== Basic Dataframe and Visualization Libraries =====

import pandas as pd                # library for data
import numpy as np                 # library for vectors
import matplotlib.pyplot as plt    # library for data
import seaborn as sns              # library for data

from functions import label_plot_edges, lat_long_plotter # function to label

# ===== Clustering and Dimensionality Reduction Libraries =====

# Clustering and Dimensionality Reduction Libraries
from sklearn.cluster import KMeans # library for K-Means
from sklearn.decomposition import PCA # library for Principal Component Analysis
from sklearn.manifold import TSNE # library for t-Distributed Stochastic Neighbor Embedding
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA # library for Linear Discriminant Analysis

import umap # library for Uniform Manifold Approximation and Projection

# Preprocessing Libraries
from sklearn.preprocessing import StandardScaler # library for feature scaling
from sklearn.preprocessing import LabelEncoder # library for encoding categorical variables
from sklearn.model_selection import train_test_split # library for splitting data

# ===== Classifier Libraries =====

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

Loading the Files as Pandas Data Frames & Global Vars

Loading the files as pandas data frames and then cleaning them up by looping through them and removing any unwanted whitespace and characters.

File	AS	Variable
gov_envData10dts_2025.csv	as	g_ed

File	AS	Variable
gov_envData10dtsMetrics_2026.csv	as	g_edm
seth_ecoRegionData_2025.csv	as	s_erd
seth_envData_2025.csv	as	s_ed
seth_envGenusCountData_2025.csv	as	s_egcd
seth_fishCountData_2025.csv	as	s_fcd
seth_genusCountData_2025.csv	as	s_gcd
seth_gov_envData10dtsMetrics_genusCount2026.csv	as	sg_edmgc

Creating the Global Variables

Variable	Type	Function
saveAll	Bool	Set to True to save all Figures

```
In [3]: g_ed = pd.read_csv('../data/gov_envData10dts_2025.csv')
g_edm = pd.read_csv('../data/gov_envData10dtsMetrics_2026.csv')
s_erd = pd.read_csv('../data/seth_ecoRegionData_2025.csv')
s_ed = pd.read_csv('../data/seth_envData_2025.csv')
s_egcd = pd.read_csv('../data/seth_envGenusCountData_2025.csv')
s_fcd = pd.read_csv('../data/seth_fishCountData_2025.csv')
s_gcd = pd.read_csv('../data/seth_genusCountData_2025.csv')
sg_edmgc = pd.read_csv('../data/seth_gov_envData10dtsMetrics_genusCount2026.csv')

# ===== Put all DataFrames in a list for batch processing =====
datasets = [g_ed, g_edm, s_erd, s_ed, s_egcd, s_fcd, s_gcd, sg_edmgc]

# ===== Clean all column names =====

for df in datasets:
    df.columns = (
        df.columns
        .str.strip()                # remove leading/trailing spaces
        .str.replace(' ', '_')      # replace internal spaces with underscore
        .str.replace('\n', '', regex=False) # remove newlines
    )

# ===== Create Global Variables =====

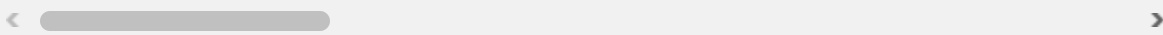
saveAll = False                    # Set to True to save all figures,
```

```
In [4]: # g_ed.info()                # check the structure of the data frame
g_ed.head()                        # printing the first 5 rows of the data frame
```

Out[4]:

	Segment_10Day	site_0.burn	site_0.dfm1000hr	site_0.dfm100hr	site_0.energy_ERC	site_0
0	0	23.4	20.15	21.12	18.1	
1	1	25.5	19.36	17.29	24.5	
2	2	25.3	18.09	15.66	28.3	
3	3	30.9	17.65	16.91	29.1	
4	4	16.2	17.47	17.54	23.8	

5 rows × 829 columns

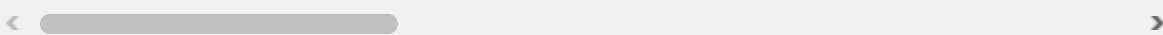


In [5]: `# g_edm.info()` *# check the structure of the data frame*
`g_edm.head()` *# printing the first 5 rows of the data frame*

Out[5]:

	site	burn_mean	burn_median	burn_sd	burn_min	burn_max	burn_range	burn_cv
0	site_1	24.772945	24.40	8.686572	5.7	57.2	51.5	0.350648
1	site_2	24.673630	24.25	8.591869	4.3	57.3	53.0	0.348221
2	site_3	25.542466	25.20	8.994322	3.0	59.1	56.1	0.352132
3	site_4	26.318836	26.30	9.289094	2.7	62.3	59.6	0.352945
4	site_5	24.677854	24.30	8.547560	5.5	55.3	49.8	0.346366

5 rows × 145 columns



In [6]: `# s_erd.info()` *# check the structure of the data frame*
`s_erd.head()` *# printing the first 5 rows of the data frame*

Out[6]:

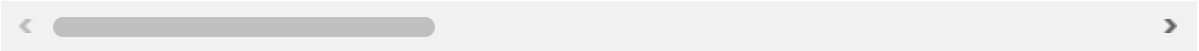
	SiteN	eco_Number	eco_Letter	Title	State
0	1	33	a	Northern Post Oak Savana	TX
1	2	33	a	Northern Post Oak Savana	TX
2	3	33	a	Northern Post Oak Savana	TX
3	4	32	a	Northern Black Land Prairie	TX
4	5	35	c	Pleistocene Fluvial Terraces	TX

In [7]: `# s_ed.info()` *# check the structure of the data frame*
`s_ed.head()` *# printing the first 5 rows of the data frame*

Out[7]:

	SiteN	State	Lat	Long	DO	pH	avg_WW	avg_Depth	pf_Pool	pf_
0	1	TX	33.741217	-95.927376	10.7	6.99	5.76666	23.75000	0.000000	83.333
1	2	TX	33.717896	-95.911323	4.2	7.22	2.94000	27.00000	60.000000	20.000
2	3	TX	33.747466	-96.019146	7.4	7.58	11.21666	46.33333	66.666667	33.333
3	4	TX	33.554952	-96.169601	7.7	7.86	10.51666	18.41666	0.000000	66.666
4	5	TX	33.823594	-95.861067	6.8	7.42	22.00000	39.66666	100.000000	0.000

5 rows × 26 columns

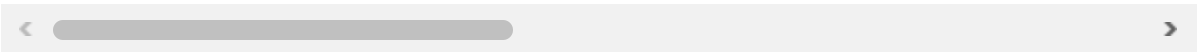


```
In [8]: # s_egcd.info()      # check the structure of the data frame
        s_egcd.head()    # printing the first 5 rows of the data frame
```

Out[8]:

	SiteN	State	Lat	Long	DO	pH	avg_WW	avg_Depth	pflow_Pool	pflow
0	1	TX	33.741217	-95.927376	10.7	6.99	5.76666	23.75000	0.000000	83.333
1	2	TX	33.717896	-95.911323	4.2	7.22	2.94000	27.00000	60.000000	20.000
2	3	TX	33.747466	-96.019146	7.4	7.58	11.21666	46.33333	66.666667	33.333
3	4	TX	33.554952	-96.169601	7.7	7.86	10.51666	18.41666	0.000000	66.666
4	5	TX	33.823594	-95.861067	6.8	7.42	22.00000	39.66666	100.000000	0.000

5 rows × 58 columns

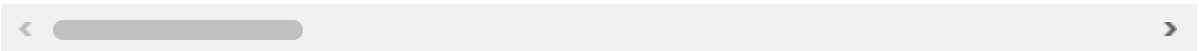


```
In [9]: # s_fcd.info()      # check the structure of the data frame
        s_fcd.head()      # printing the first 5 rows of the data frame
```

Out[9]:

	SiteN	State	Etheostoma_radiosum	Lepomis_megalotis	Lepomis_macrochirus	Lepomis_g...
0	1	1		0	6	5
1	2	1		0	0	0
2	3	1		0	24	12
3	4	1		0	43	54
4	5	1		0	16	16

5 rows × 54 columns



```
In [10]: # s_gcd.info()      # check the structure of the data frame
         s_gcd.head()      # printing the first 5 rows of the data frame
```

Out[10]:

	SiteN	State	Ameiurus	Aphredoderus	Aplodinotus	Campostoma	Cyprinella	Cyprinus
0	1	TX	0	0	0	1	0	0
1	2	TX	0	0	0	0	0	0
2	3	TX	0	0	0	0	0	0
3	4	TX	0	0	0	36	39	1
4	5	TX	0	0	1	0	22	0

5 rows × 32 columns

In [11]:

```
# sg_edmgc.info()      # check the structure of the data frame
sg_edmgc.head()       # printing the first 5 rows of the data frame
```

Out[11]:

	SiteN	State	Lat	Long	DO	pH	avg_WW	avg_Depth	pflow_Pool	pflow
0	1	TX	33.741217	-95.927376	10.7	6.99	5.76666	23.75000	0.000000	83.33
1	2	TX	33.717896	-95.911323	4.2	7.22	2.94000	27.00000	60.000000	20.00
2	3	TX	33.747466	-96.019146	7.4	7.58	11.21666	46.33333	66.666667	33.33
3	4	TX	33.554952	-96.169601	7.7	7.86	10.51666	18.41666	0.000000	66.66
4	5	TX	33.823594	-95.861067	6.8	7.42	22.00000	39.66666	100.000000	0.00

5 rows × 202 columns

Bar Plots

In this Section I will generate encompassing bar plots to get good overviews of the # of fish species, genus, site attributes and metrics.

I will hope to encompass any relevant but overwhelming data into basic bar plots to get a simple start on understanding this data visually and identify any trends or under-represented features.

Total Fish Species Bar Plot

This cell simply sums all of the fish species columns in the **seth_fishCountData_2025.csv** dataset and visualizes it as a bar plot.

Here I am looking to see the distribution of species so that I can determine species is abundance and few. My hope is that this will be useful in

determining outliers later, and possibly visualize species in fewer numbers than expected.

```
In [12]: # ===== Data Analysis =====

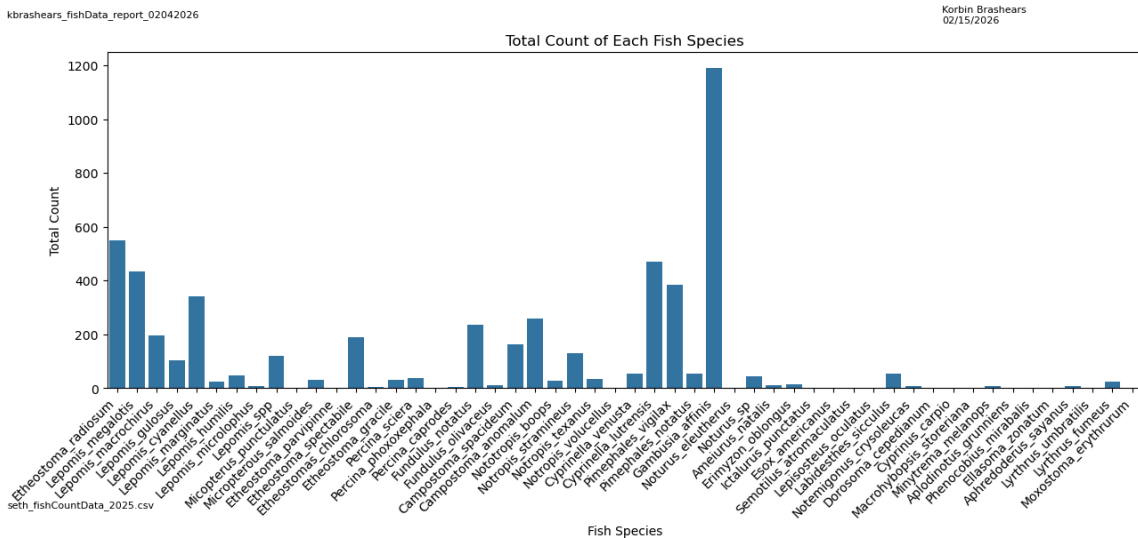
s_fcd_species_cols = s_fcd.columns[2:] # selecting
fish_species_sum = s_fcd[s_fcd_species_cols].sum() # summing e

# ===== Plotting =====

plt.figure(figsize=(15, 6)) # creating
sns.barplot(x=fish_species_sum.index, y=fish_species_sum.values) # creating
plt.xticks(rotation=45, ha='right') # rotating
plt.xlabel('Fish Species') # setting x
plt.ylabel('Total Count') # setting y
plt.title('Total Count of Each Fish Species') # setting t

label_plot_edges(
    program_names="kbrashears_fishData_report_02042026", # calling t
    author_names="Korbin Brashears",
    data_filenames="seth_fishCountData_2025.csv",
    fig_filename="./figures/kbrashears_fishData_report_02042026/fish_species_barPl
    save = saveAll if saveAll else False # change Fa
)

plt.show() # displaying
```



Total Fish Genus Bar Plot

This cell preforms the same action as before on the **seth_genusCountData_2025.csv**, summing the columns and visualizing as a bar plot.

Hopefully this can help to find outliers with unexpected differences between the population in an over all genus vs a species.

```

In [13]: # ===== Data Analysis =====

s_gcd_genus_cols = s_gcd.columns[2:] # selecting
fish_genus_sum = s_gcd[s_gcd_genus_cols].sum() # summing e

# ===== Plotting =====

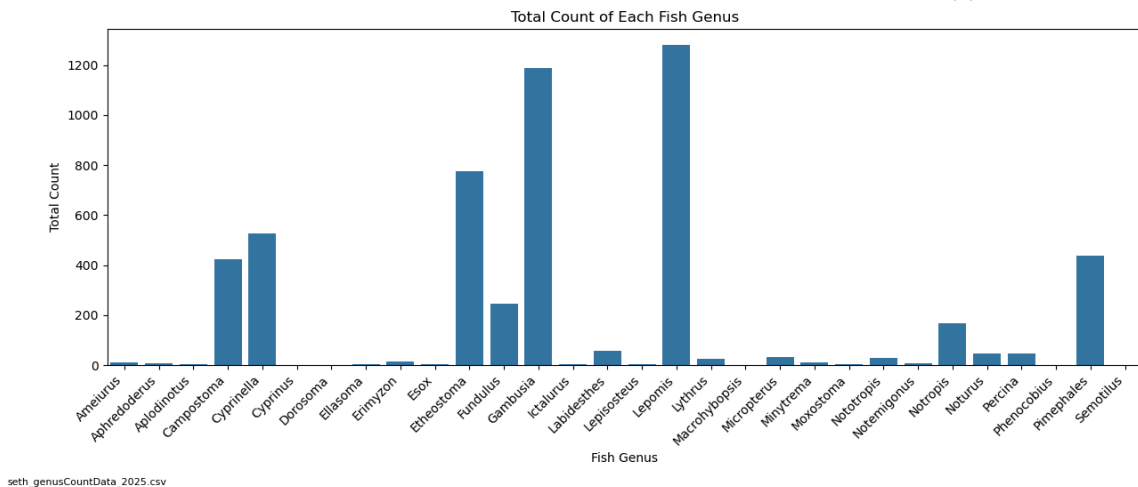
plt.figure(figsize=(15, 6)) # creating
sns.barplot(x=fish_genus_sum.index, y=fish_genus_sum.values) # creating
plt.xticks(rotation=45, ha='right') # rotating
plt.xlabel('Fish Genus') # setting x
plt.ylabel('Total Count') # setting y
plt.title('Total Count of Each Fish Genus') # setting t

label_plot_edges(
    program_names="kbrashears_fishData_report_02042026", # calling t
    author_names="Korbin Brashears",
    data_filenames="seth_genusCountData_2025.csv",
    fig_filename=" ../figures/kbrashears_fishData_report_02042026/fish_genus_barPlot
    save = saveAll if saveAll else False # change Fa
)

plt.show() # displayin

```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_genusCountData_2025.csv

Total Population and Genus at Each Site

```

In [14]: # ===== Plotting =====

plt.figure(figsize=(15, 8)) # creating

plt.subplot(211)
sns.barplot(x=s_egcd['SiteN'], y=s_egcd['t_population']) # creating
plt.xticks(rotation=45, ha='right') # rotating
plt.xlabel('Site Number') # setting x
plt.ylabel('Total Fish Population') # setting y

```



```

plt.title('Total Fish Population at Each Site') # setting t

plt.subplot(212)
sns.barplot(x=s_egcd['SiteN'], y=s_egcd['t_genus']) # creating
plt.xticks(rotation=45, ha='right') # rotating
plt.xlabel('Site Number') # setting x
plt.ylabel('Total Fish Genus Count') # setting y
plt.title('Total Fish Genus Count at Each Site') # setting t

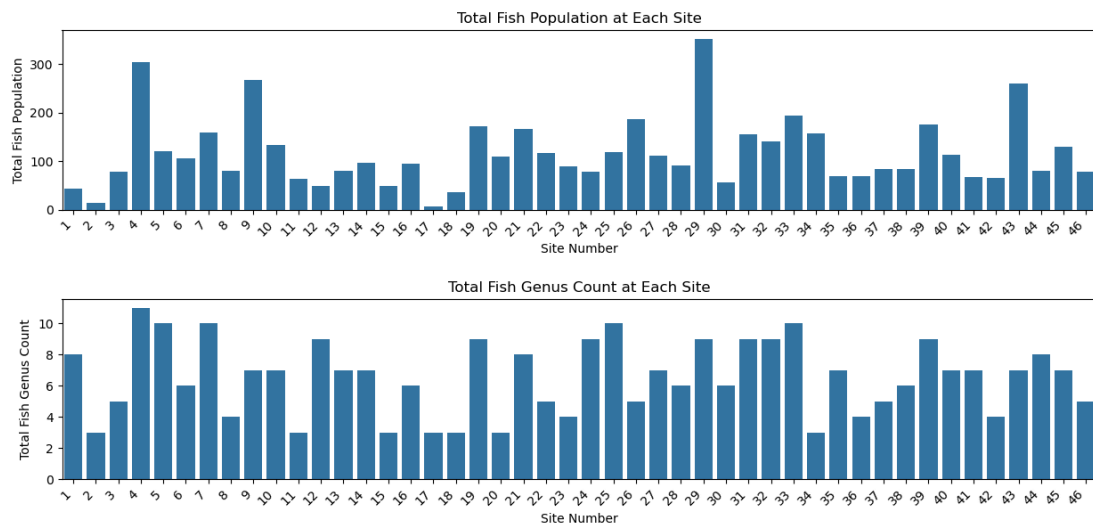
plt.subplots_adjust(hspace=0.5) # adjusting

label_plot_edges(
    program_names="kbrashears_fishData_report_02042026", # calling t
    author_names="Korbin Brashears",
    data_filenames="seth_envGenusCountData_2025.csv",
    fig_filename=" ../figures/kbrashears_fishData_report_02042026/fish_population_ba
    save = saveAll if saveAll else False # change Fa
)

plt.show() # displayin

```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envGenusCountData_2025.csv

Total Population and Genus at Each Site Ordered Bar Plot

```

In [15]: # ===== Data Analysis =====

s_egcd_tps = s_egcd.sort_values(by='t_population', ascending=False) # sorting t
s_egcd_tgs = s_egcd.sort_values(by='t_genus', ascending=False) # sorting t

# ===== Plotting =====

plt.figure(figsize=(15, 8)) # creating

plt.subplot(211)

```

```

sns.barplot(x=s_egcd_tps['SiteN'], y=s_egcd_tps['t_population'], order=s_egcd_tps['
plt.xticks(rotation=45, ha='right') # rotating
plt.xlabel('Site Number') # setting x
plt.ylabel('Total Fish Population') # setting y
plt.title('Total Fish Population at Each Site') # setting t

plt.subplot(212)
sns.barplot(x=s_egcd_tgs['SiteN'], y=s_egcd_tgs['t_genus'], order=s_egcd_tgs['SiteN
plt.xticks(rotation=45, ha='right') # rotating
plt.xlabel('Site Number') # setting x
plt.ylabel('Total Fish Genus Count') # setting y
plt.title('Total Fish Genus Count at Each Site') # setting t

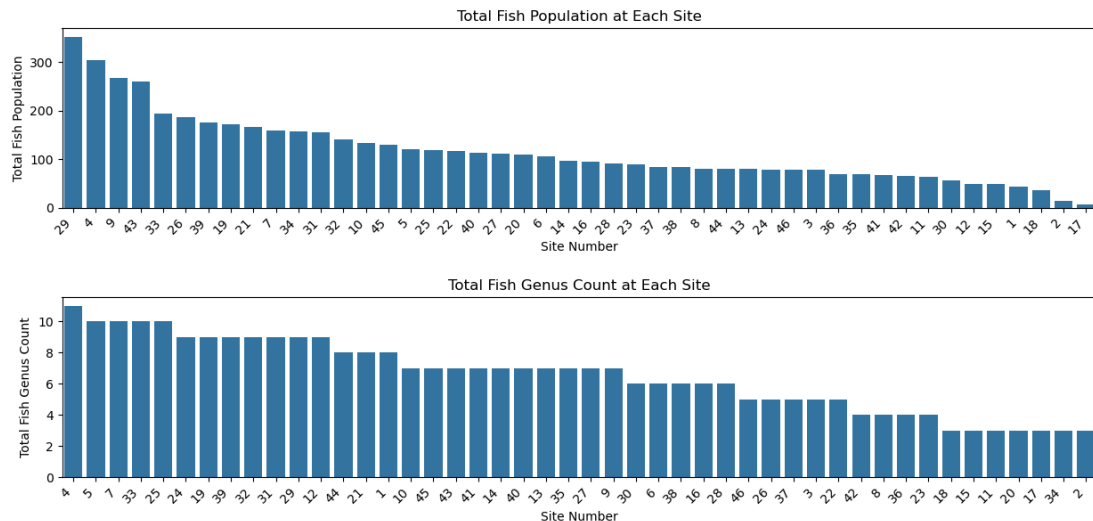
plt.subplots_adjust(hspace=0.5) # adjusting

label_plot_edges(
    program_names="kbrashears_fishData_report_02042026", # calling t
    author_names="Korbin Brashears",
    data_filenames="seth_envGenusCountData_2025.csv",
    fig_filename=" ../figures/kbrashears_fishData_report_02042026/fish_population_ba
    save = saveAll if saveAll else False # change Fa
)

plt.show() # displayin

```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envGenusCountData_2025.csv

Site Attribute Bar Plots

A function for selecting and summing the attribute columns from the file **seth_envData_2025.csv** for a given site N. This will help to see the over all site attribute distribution at a site of interest and manually spot similarities or outliers.

```

In [16]: # ===== Defining Function =====

def site_attributes_bar_plotter(siteN, save=False):

    # ===== Data Analysis =====

    # Row Selection
    site_att_v = s_ed[siteN-1:siteN]

    att_cols = site_att_v.columns[4:]
    site_att_v = site_att_v[att_cols]

    # Normalization
    max_val = site_att_v.values.max()
    site_att_v = site_att_v / max_val

    # Sorting
    site_att_ov = site_att_v.T.sort_values(by=siteN-1, ascending=False).T

    # ===== Plotting =====

    plt.figure(figsize=(10, 10))

    #-----
    # Subplot 1: Unordered Bar Plot
    #-----

    # Creating Subplot
    plt.subplot(211)

    # Creating Bar Plot
    sns.barplot(x=site_att_v.columns, y=site_att_v.values[0])

    # Setting Lables and Rotation
    plt.xticks(rotation=45, ha='right')
    plt.xlabel('')
    plt.ylabel('Attribute Value')

    # Setting Title
    plt.title(f'Environmental Attributes for Site #{siteN}')

    #-----
    # Subplot 2: Ordered Bar Plot
    #-----

    # Creating Subplot
    plt.subplot(212)

    # Creating Bar Plot
    sns.barplot(x=site_att_ov.columns, y=site_att_ov.values[0])

```

```

# Setting Lables and Rotation
plt.xticks(rotation=45, ha='right')
plt.xlabel('Environmental Attributes')
plt.ylabel('Attribute Value')

# Setting Title
plt.title(f'Ordered Environmental Attributes for Site #{siteN}')

# Adding Vertical Space Between Subplots
plt.subplots_adjust(hspace=0.75)

# ===== Adding Plot Edge Labels =====

label_plot_edges(
    program_names="kbrashears_fishData_report_02042026",
    author_names="Korbin Brashears",
    data_filenames="seth_envData_2025.csv",
    fig_filename=f"../figures/kbrashears_fishData_report_02042026/site_{siteN}_",
    save = saveAll if saveAll else save
)

plt.show()

return None

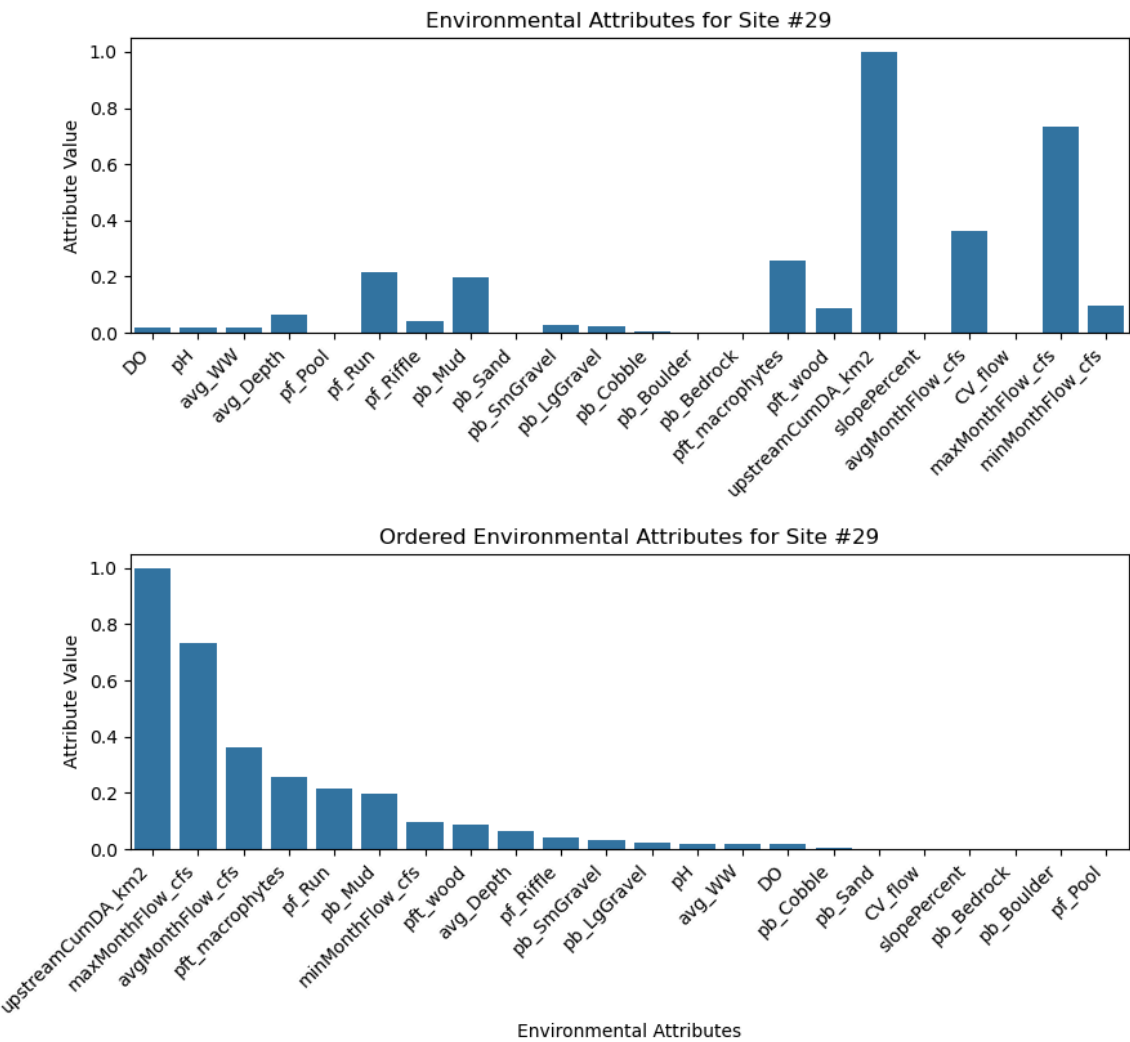
```

High t_{population} Sites

Attribute bar plots for the top 4 t_{population} sites in order (high to low t_{pop})

Site #29 Bar Plots

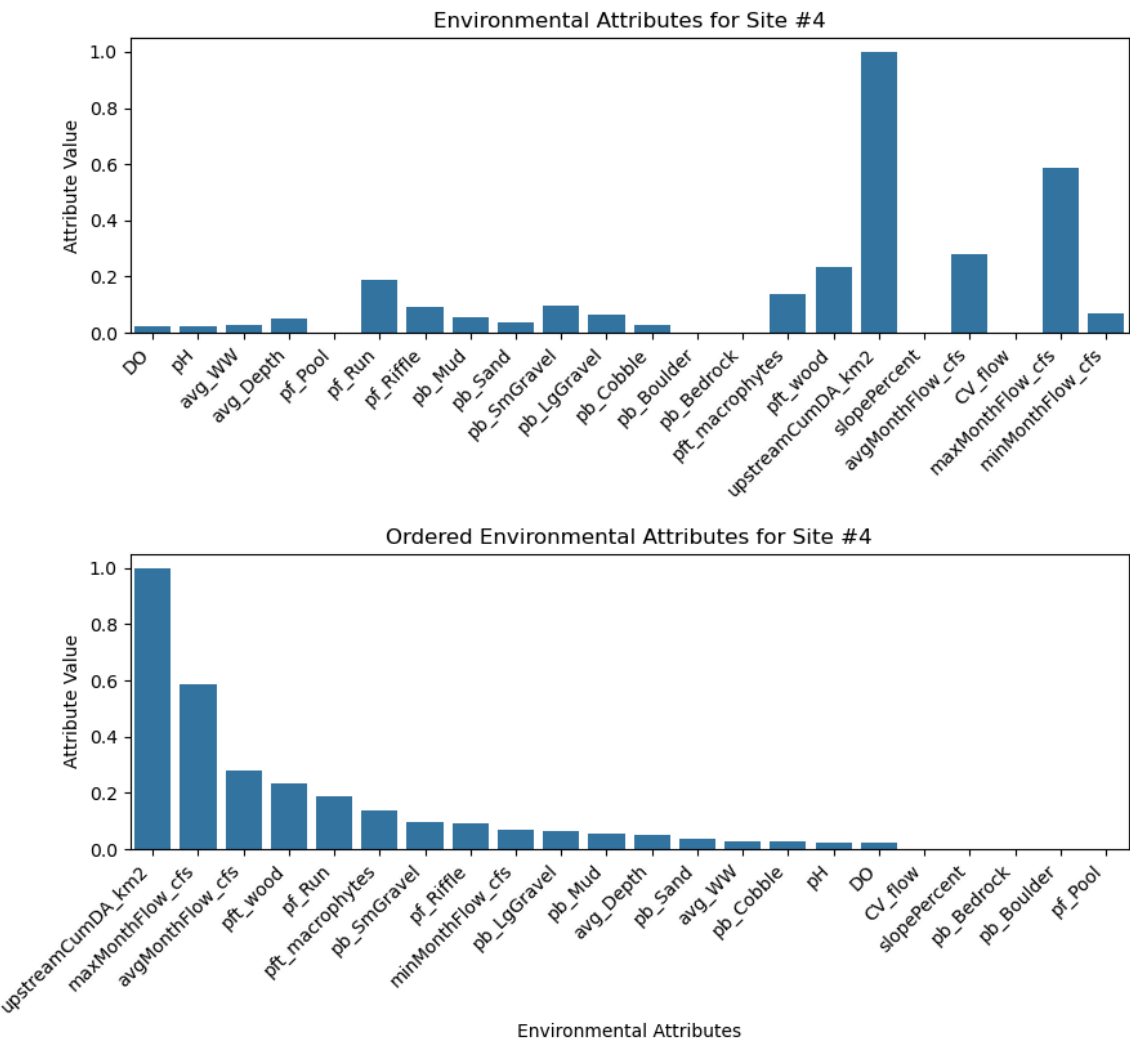
```
In [17]: site_attributes_bar_plotter(siteN=29, save=False) # calling the function to c
```



seth_envData_2025.csv

Site #4 Bar Plots

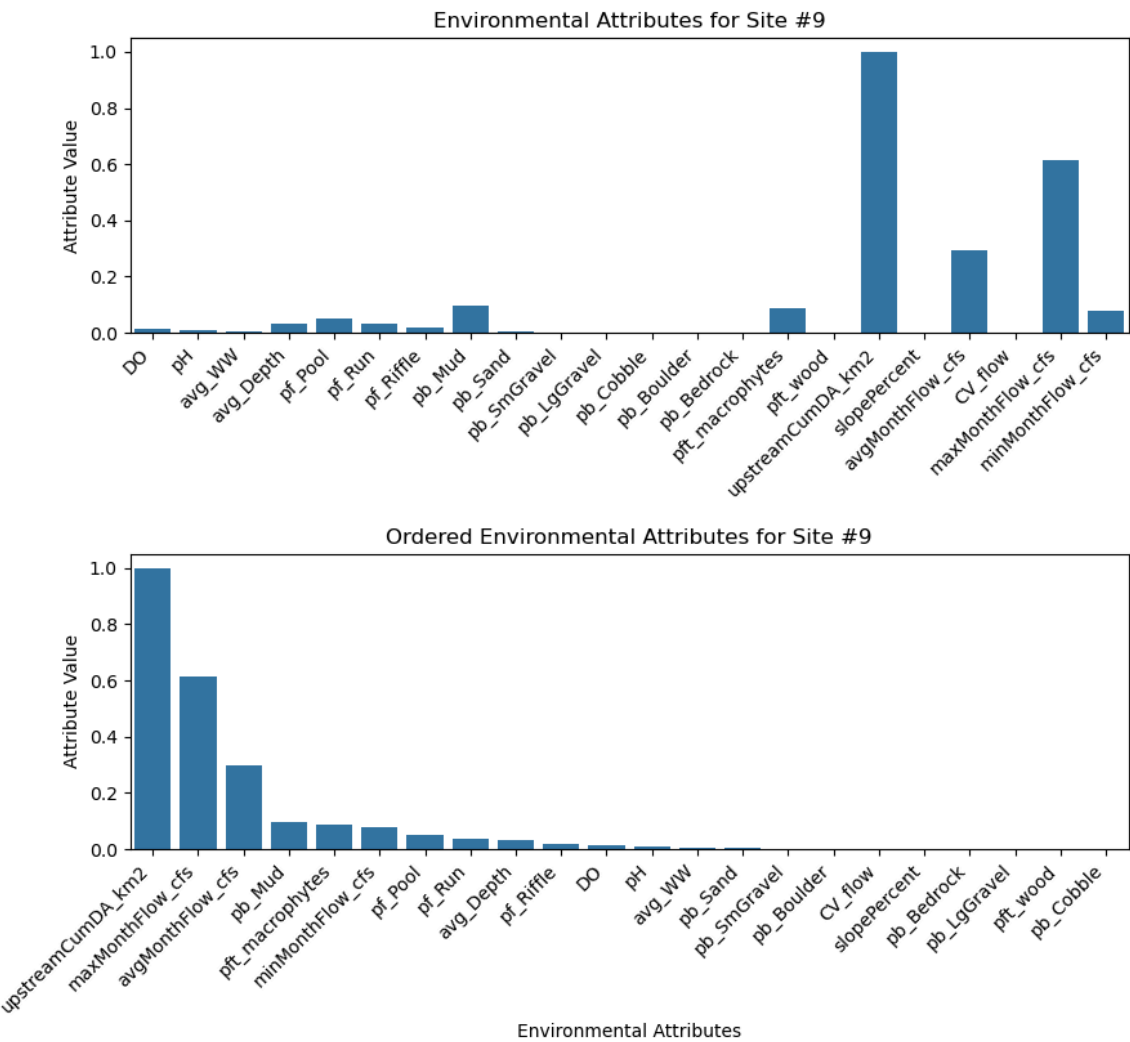
```
In [18]: site_attributes_bar_plotter(siteN=4, save=False) # calling the function to cr
```



seth_envData_2025.csv

Site #9 Bar Plots

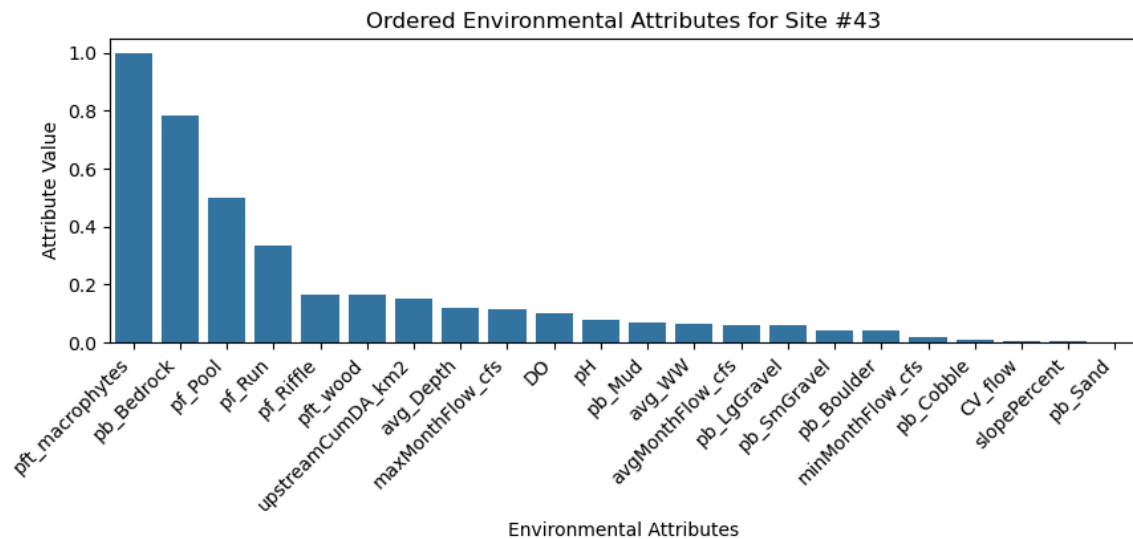
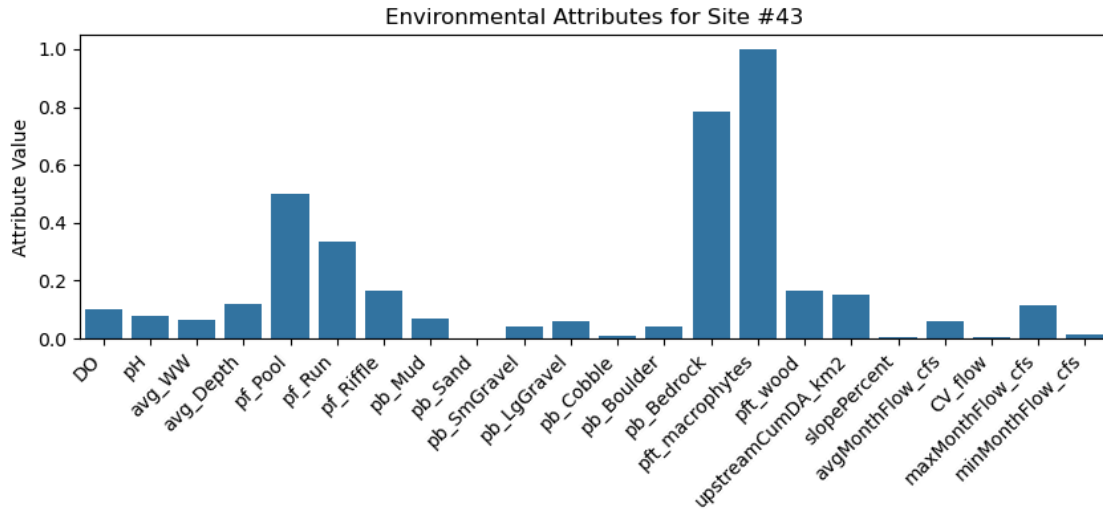
```
In [19]: site_attributes_bar_plotter(siteN=9, save=False) # calling the function to cr
```



seth_envData_2025.csv

Site #43 Bar Plots

```
In [20]: site_attributes_bar_plotter(siteN=43, save=False) # calling the function to c
```



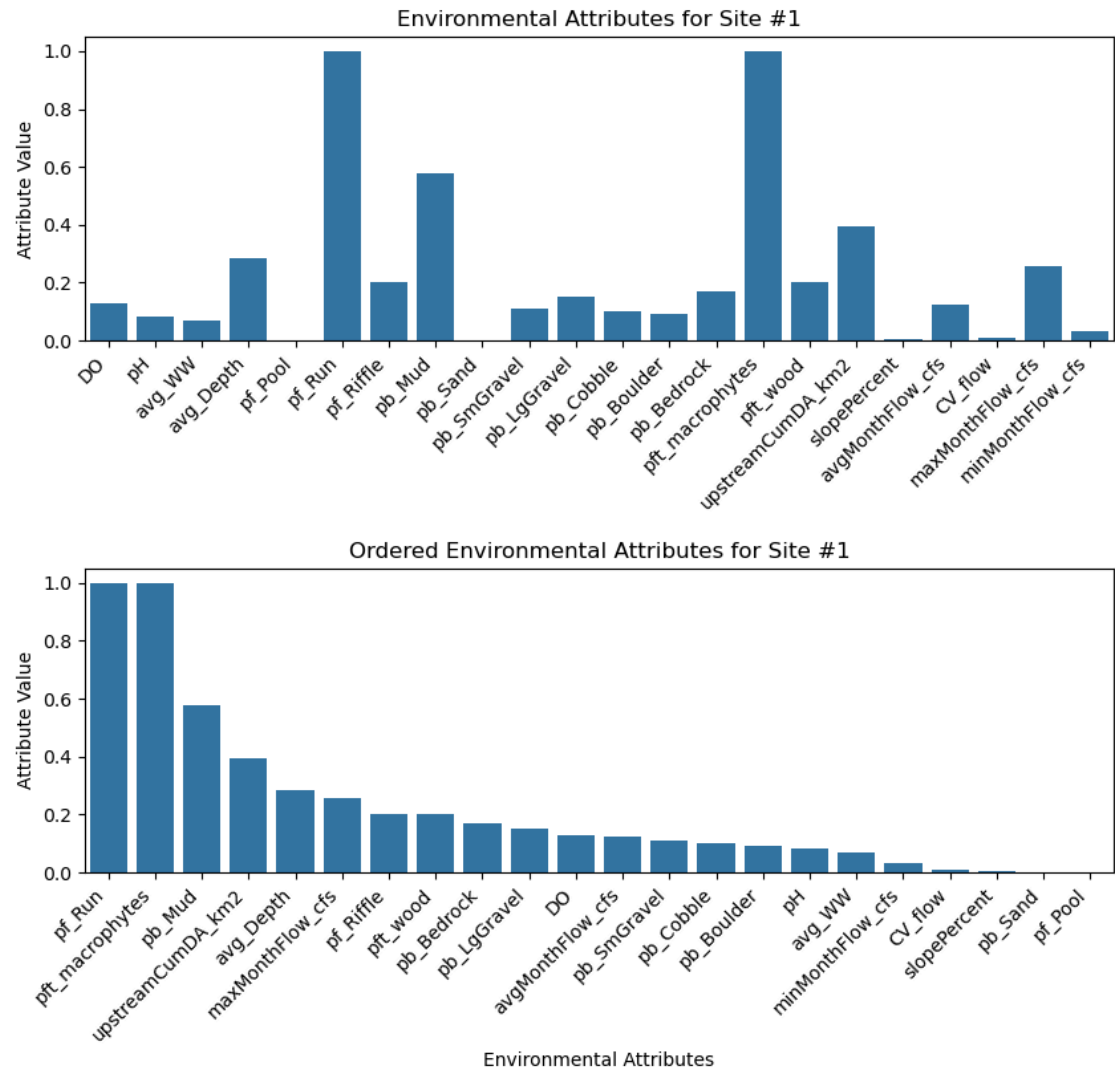
seth_envData_2025.csv

Low t_population Sites

Attribute bar plots for the bottom 4 t_population sites in order (high to low t_pop)

Site #1 Bar Plots

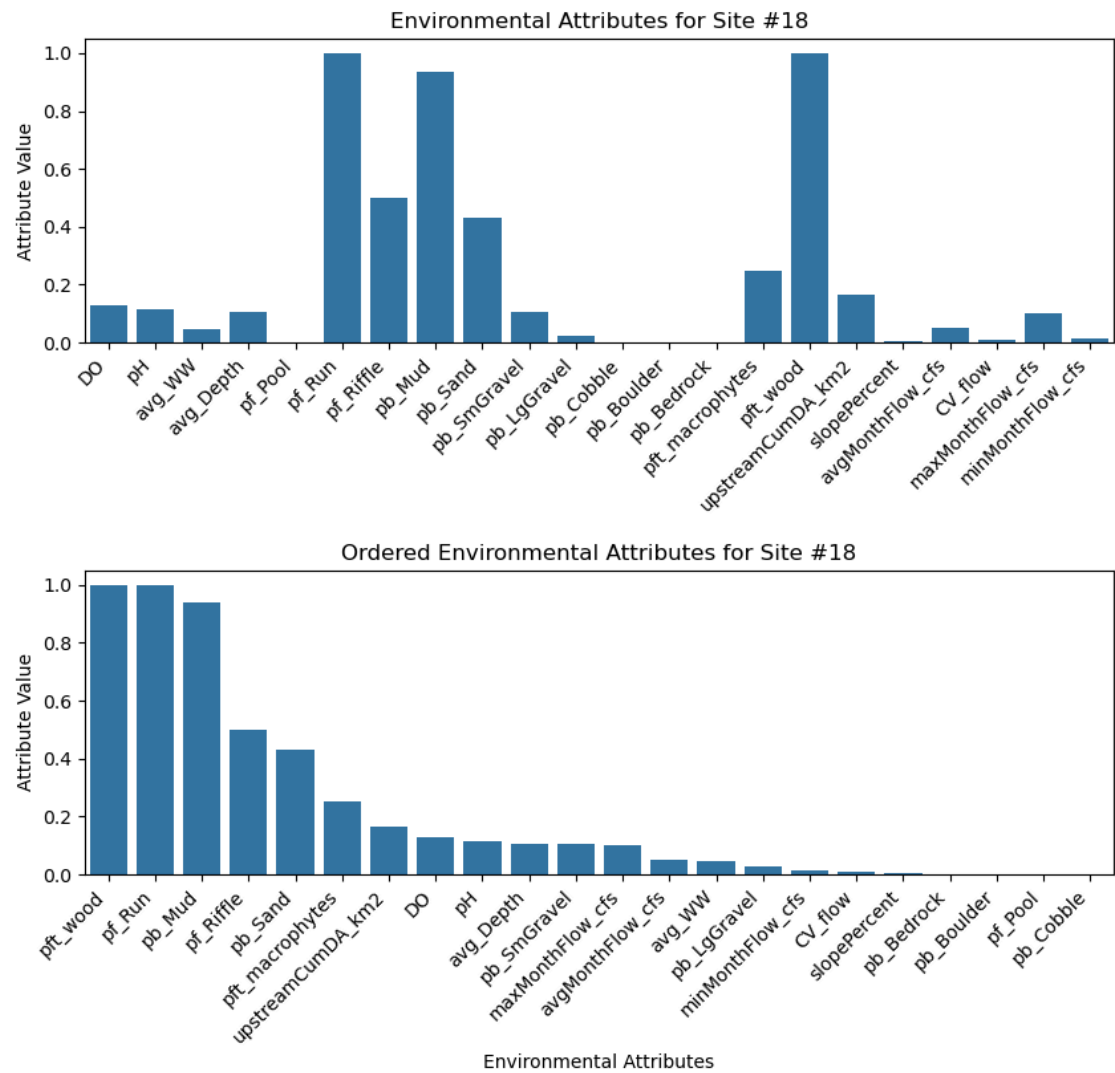
```
In [21]: site_attributes_bar_plotter(siteN=1, save=False) # calling the function to cr
```

seth_envData_2025.csv

Site #18 Bar Plots

```
In [22]: site_attributes_bar_plotter(siteN=18, save=False) # calling the function to c
```

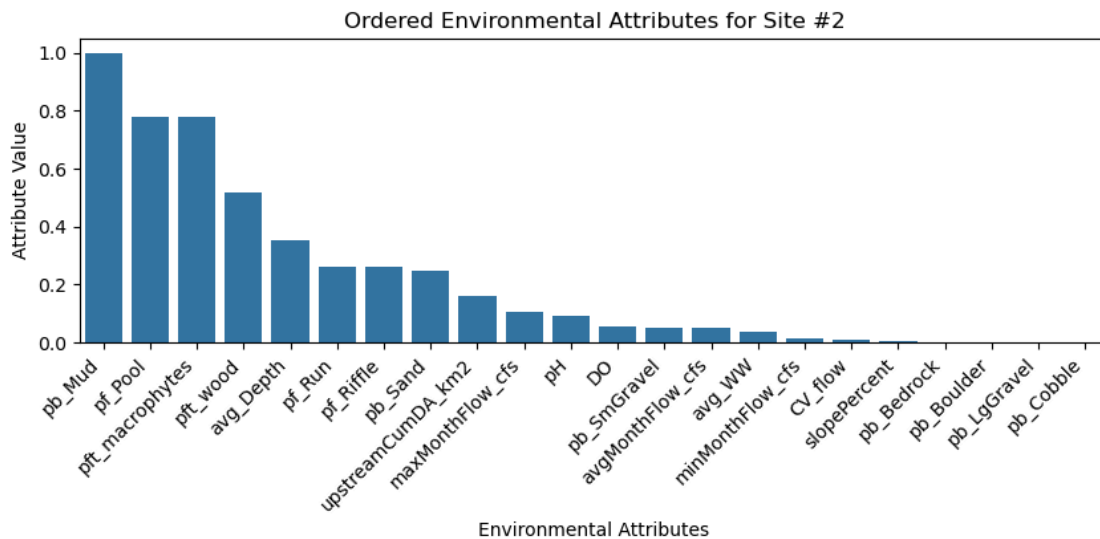
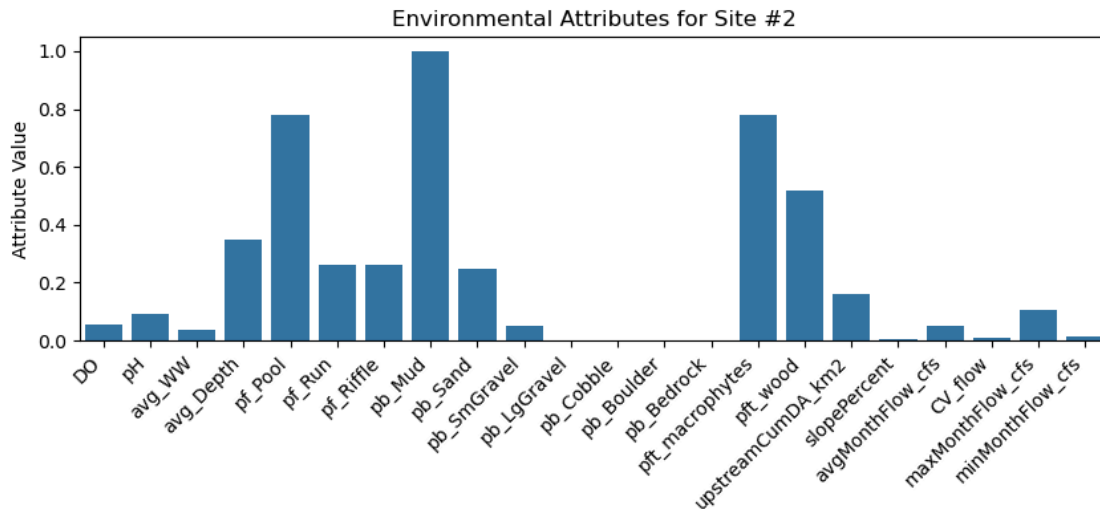


seth_envData_2025.csv

Site #2 Bar Plots

```
In [23]: site_attributes_bar_plotter(siteN=2, save=False) # calling the function to cr
```

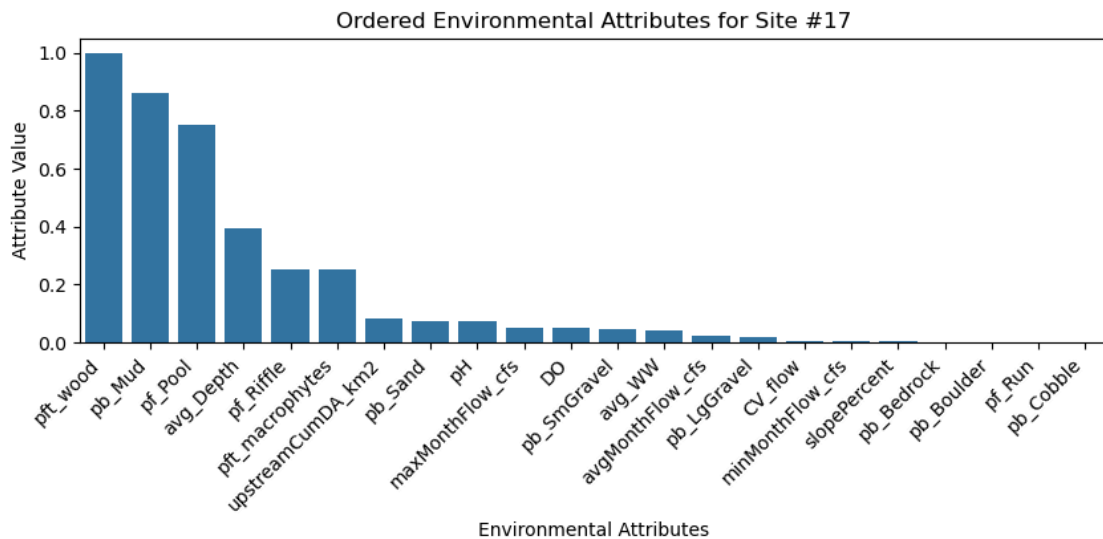
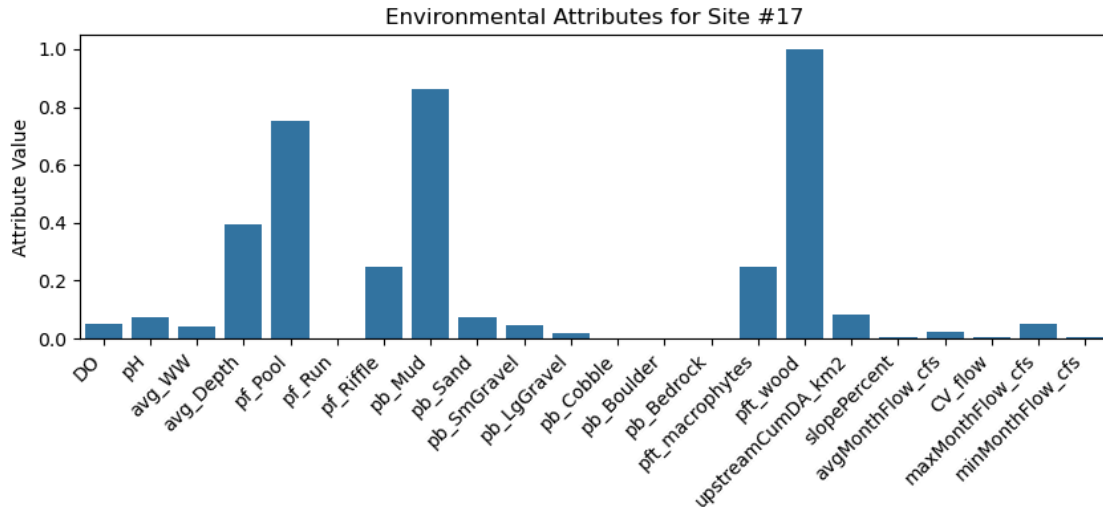
kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

Site #17 Bar Plots

```
In [24]: site_attributes_bar_plotter(siteN=17, save=False) # calling the function to c
```



seth_envData_2025.csv

Correlation Plots

seth_envData_2025.csv Correlation Heatmap

```
In [25]: # ===== Data Analysis =====

# Selecting Attribute Columns and Fish Count Columns
att_cols = list(s_egcd.columns[4:26]) + list(s_egcd.columns[56:])

# Calculating Correlation Matrix
s_ed_c_m = s_egcd[att_cols].corr(numeric_only=True)

# ===== Plotting =====
```

```
# Creating Figure
plt.figure(figsize=(12, 10))

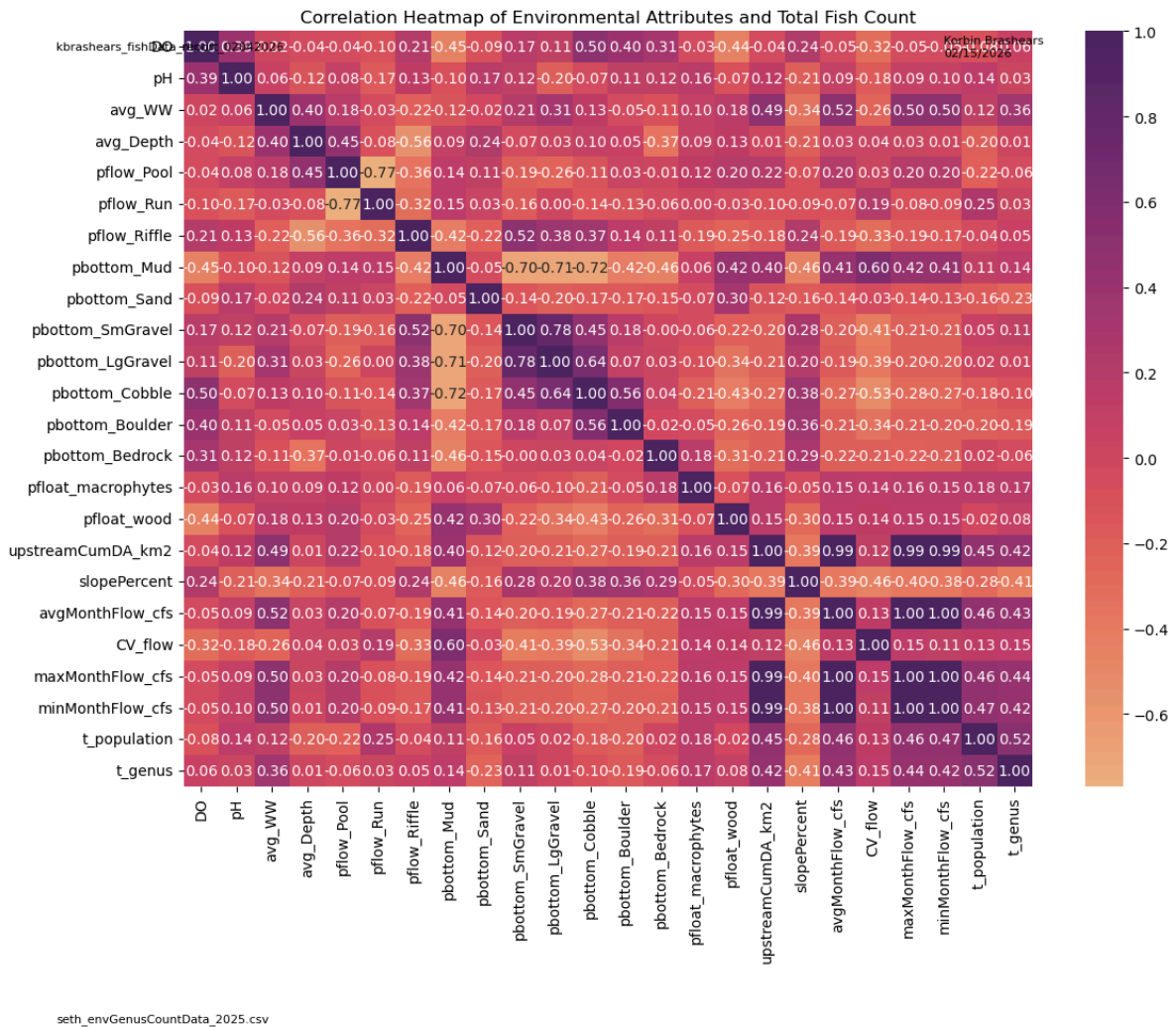
# Creating Heatmap
sns.heatmap(s_ed_c_m, annot=True, cmap='flare', fmt='.2f')

plt.title('Correlation Heatmap of Environmental Attributes and Total Fish Count')

plt.tight_layout()

label_plot_edges(
    program_names="kbrashears_fishData_report_02042026",
    author_names="Korbin Brashears",
    data_filenames="seth_envGenusCountData_2025.csv",
    fig_filename="./figures/kbrashears_fishData_report_02042026/environmental_attr",
    save = saveAll if saveAll else False
)

plt.show()
```



Ranked Correlation Decay Plot

Creating a function to take a target feature to compare against, and the features to be ranked, finds the correlations, sorts them high to low, and plots the result.

```
In [26]: # ===== Defining Function =====

def corr_decay_plotter(attribute, features, save=False):

    # ===== Data Analysis =====

    # Calculating Correlations with Respect to the Specified Attribute
    cd_raw = features.corrwith(attribute, numeric_only=True)
    neg_idx = cd_raw < 0
    cd = cd_raw.abs()

    # Sorting Correlation Values
    cd_sorted = cd.sort_values(ascending=False)
    neg_sorted = neg_idx.loc[cd_sorted.index]

    # ===== Plotting =====

    # Creating the Figure
    plt.figure(figsize=(12, 6))

    # Creating Line Plot (base Line)
    sns.lineplot(x=cd_sorted.index, y=cd_sorted.values, color='blue', zorder=1)

    # Plotting Positive Correlations
    sns.scatterplot(
        x=cd_sorted.index[~neg_sorted],
        y=cd_sorted.values[~neg_sorted],
        color="blue",
        zorder=2,
        s=40
    )

    # Plotting Negative Correlations
    sns.scatterplot(
        x=cd_sorted.index[neg_sorted],
        y=cd_sorted.values[neg_sorted],
        color="orange",
        zorder=2,
        s=40
    )

    # Setting Labels and Rotation
    plt.xticks(rotation=45, ha='right')
    plt.xlabel('')
    plt.ylabel('Correlation Coefficient')
    plt.title(f'Correlation Decay of features with {attribute.name}')

    plt.grid(True)
```

```
# ===== Adding Plot Edge Labels =====

# Labeling Plot Edges
label_plot_edges(
    program_names="kbrashears_fishData_report_02042026",
    author_names="Korbin Brashears",
    data_filenames="seth_envGenusCountData_2025.csv",
    fig_filename=f"../figures/kbrashears_fishData_report_02042026/{attribute}_c
    save = saveAll if saveAll else save
)

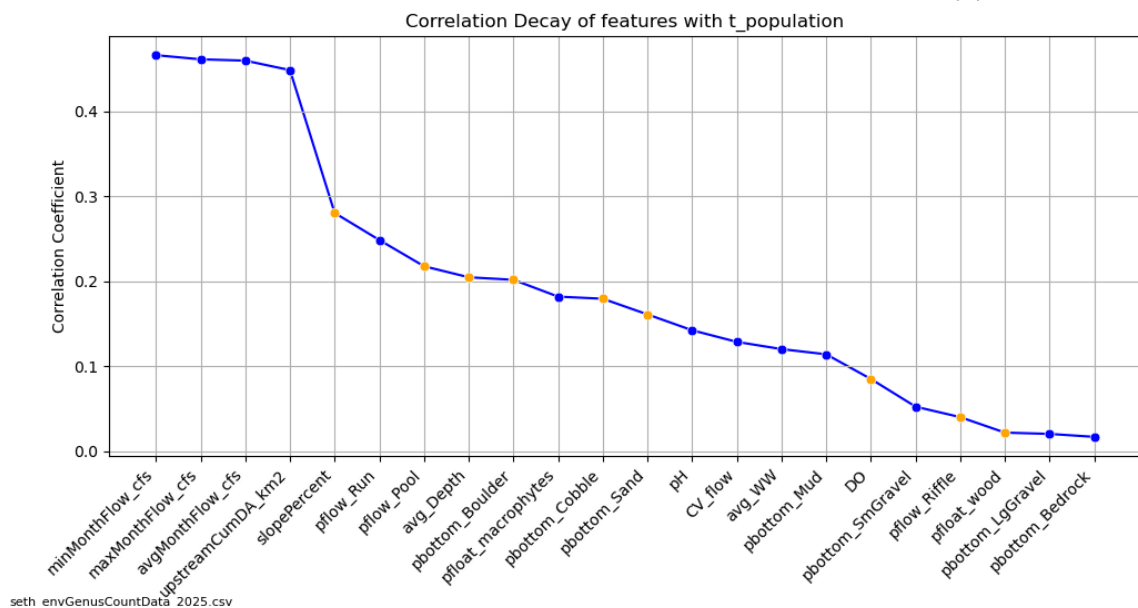
plt.show()

return None
```

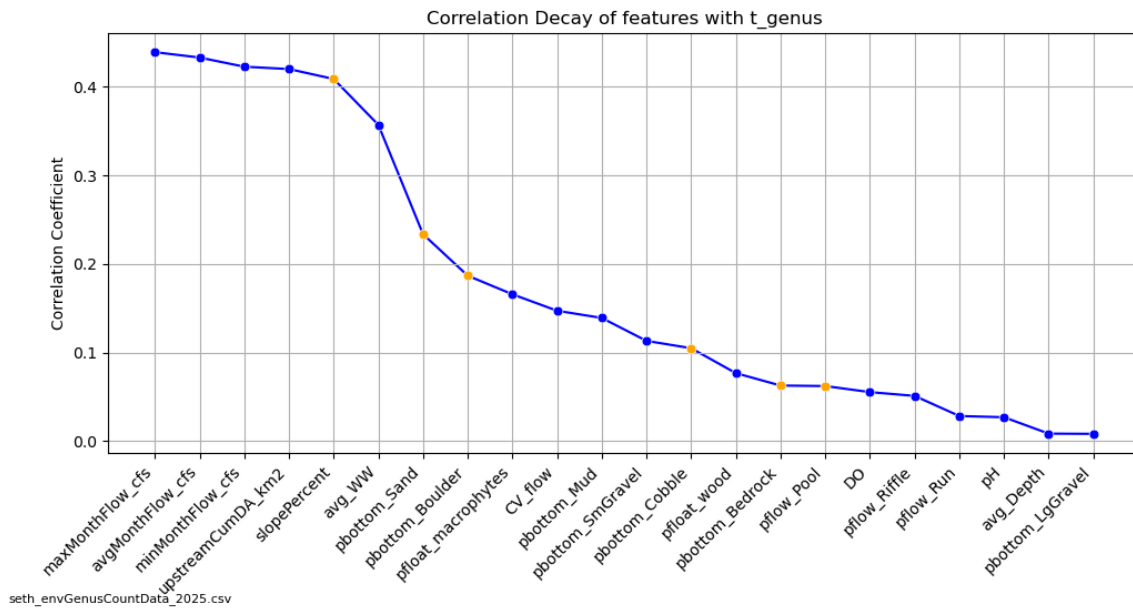
```
In [27]: att_cols = s_egcd.columns[4:26]

corr_decay_plotter(attribute=s_egcd['t_population'], features=s_egcd[att_cols], sav
corr_decay_plotter(attribute=s_egcd['t_genus'], features=s_egcd[att_cols], save=Fa
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

Scatter TrendLine Plotter

Defining a function, **scatter_trendLine_plotter()** to plot a scatter plot and trend line for x and y vectors of the same length.

```
In [28]: # ===== Defining Function =====

def scatter_trendLine_plotter(x, y, pointLabels=True, save=False):

    # ===== Plotting =====

    plt.figure(figsize=(10, 6))

    # Adding Trend Line
    sns.regplot(
        x=x,
        y=y,
        scatter=False,          # turn off regplot's own dots
        line_kws={"linewidth": 2, "linestyle": "--", "zorder": 1, "color": "#000000"}
    )

    # Creating Scatter Plot
    sns.scatterplot(
        x=x,
        y=y,
        s=60,                   # dot size
        color="#ff0000",        # custom hex color (red)
        #zorder=2,
        edgecolor="black"       # optional: outline around dots
    )

    # Adding Point Index Labels (if enabled)
    if pointLabels:
```



```

    for i in range(len(x)):
        plt.text(x[i], y[i], str(i), fontsize=9, ha='right', va='bottom', color=

# Labeling Axis and Title
plt.xlabel(x.name)
plt.ylabel(y.name)

plt.title(f'{y.name} vs {x.name}')

# ===== Adding Plot Edge Labels =====

# Adding Plot Edge Labels
label_plot_edges(
    program_names="kbrashears_fishData_report_02042026",
    author_names="Korbin Brashears",
    data_filenames="seth_envData_2025.csv",
    fig_filename=f"../figures/kbrashears_fishData_report_02042026/{x.name}_vs_{y.name}.png",
    save = saveAll if saveAll else False
)

plt.show()

return None

```

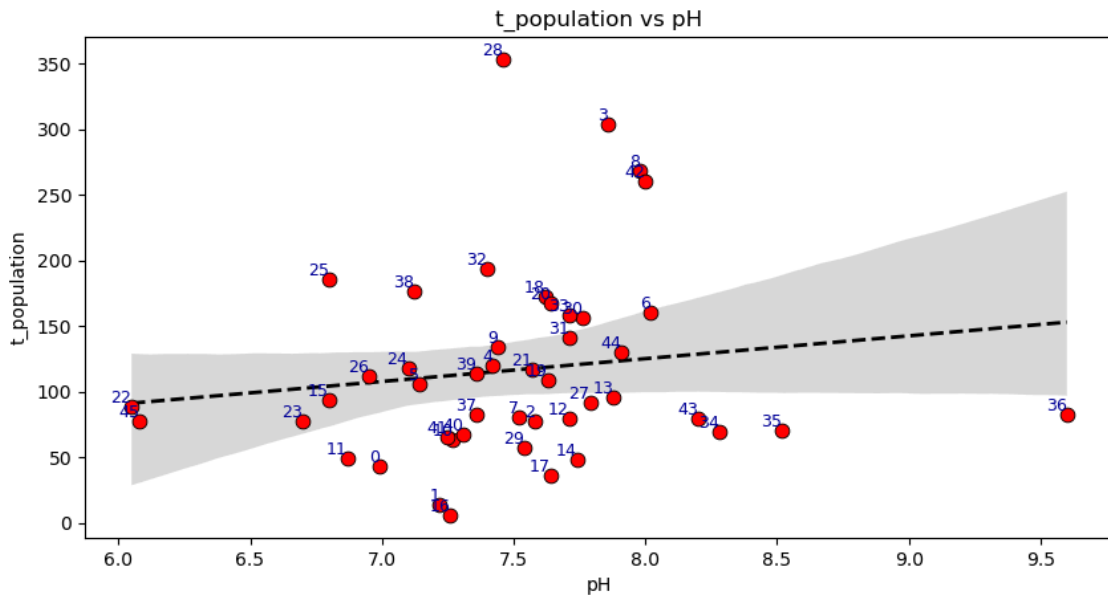
pH Scatter Plot

```

In [29]: scatter_trendLine_plotter(s_ed['pH'], s_egcd['t_population'], save=False) # c
scatter_trendLine_plotter(s_ed['pH'], s_egcd['t_genus'], save=False) # c

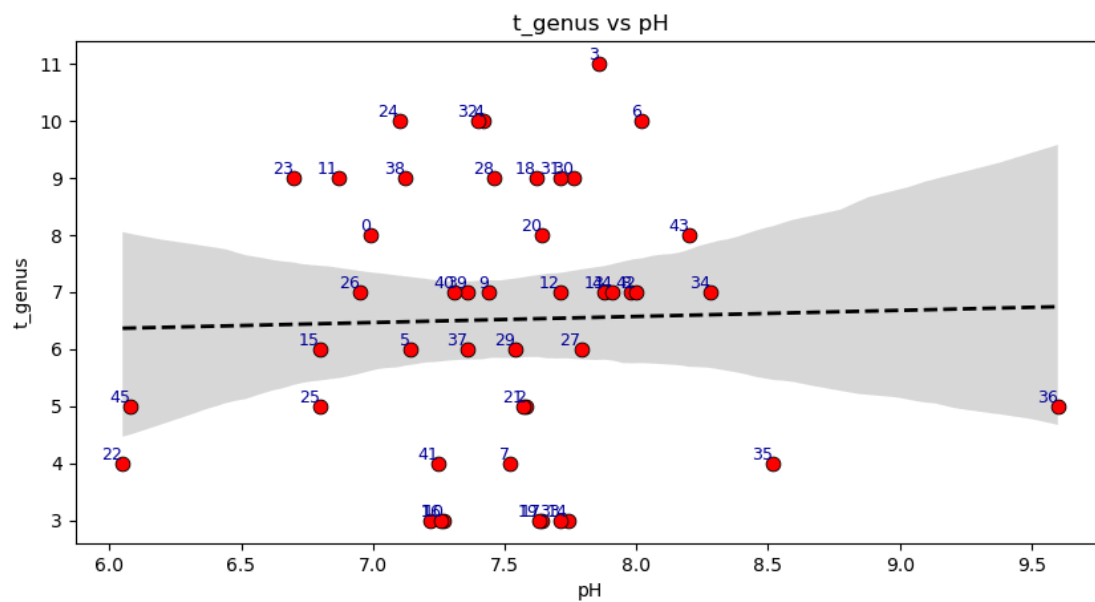
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

kbrashears_fishData_report_02042026

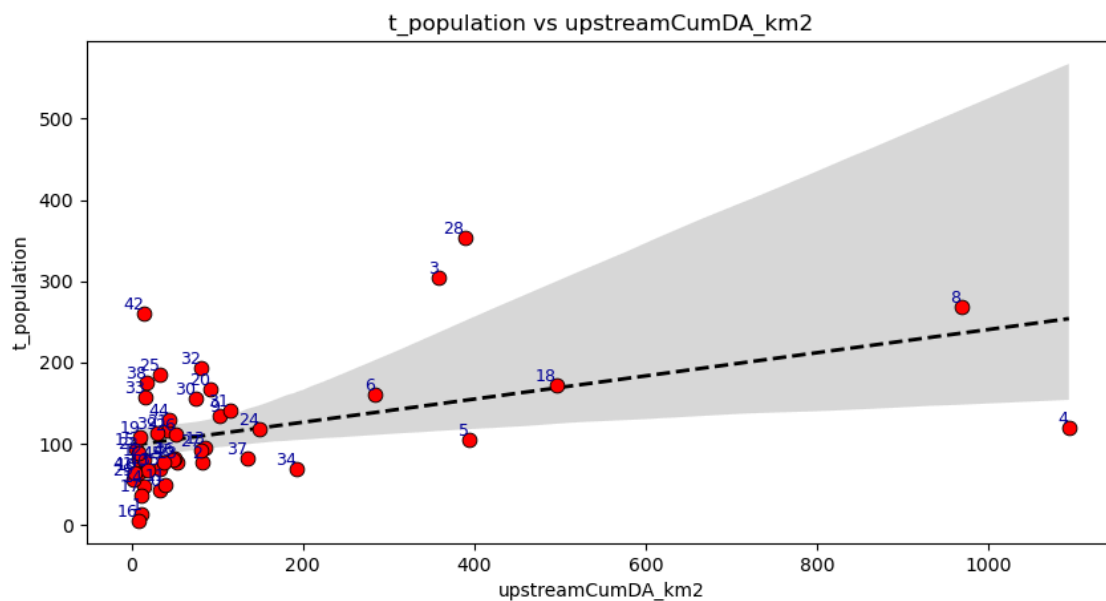
Korbin Brashears
02/15/2026

seth_envData_2025.csv

upstreamCumDA_km2 Scatter Plot

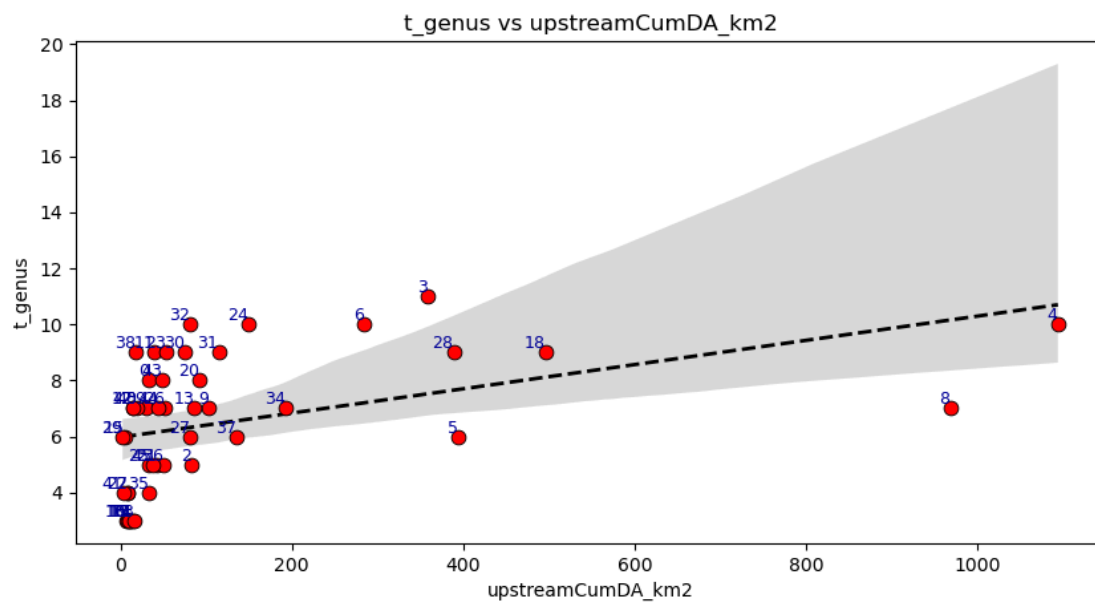
```
In [30]: scatter_trendLine_plotter(s_ed['upstreamCumDA_km2'], s_egcd['t_population'], save=False)
scatter_trendLine_plotter(s_ed['upstreamCumDA_km2'], s_egcd['t_genus'], save=False)
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

kbrashears_fishData_report_02042026

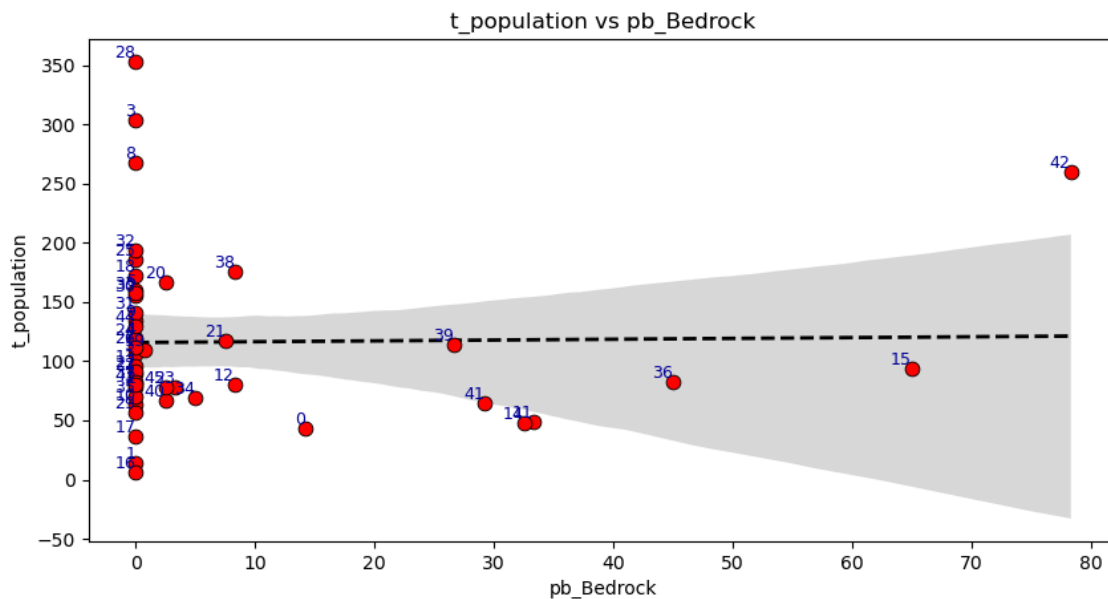
Korbin Brashears
02/15/2026

seth_envData_2025.csv

pb_Bedrock Scatter Plot

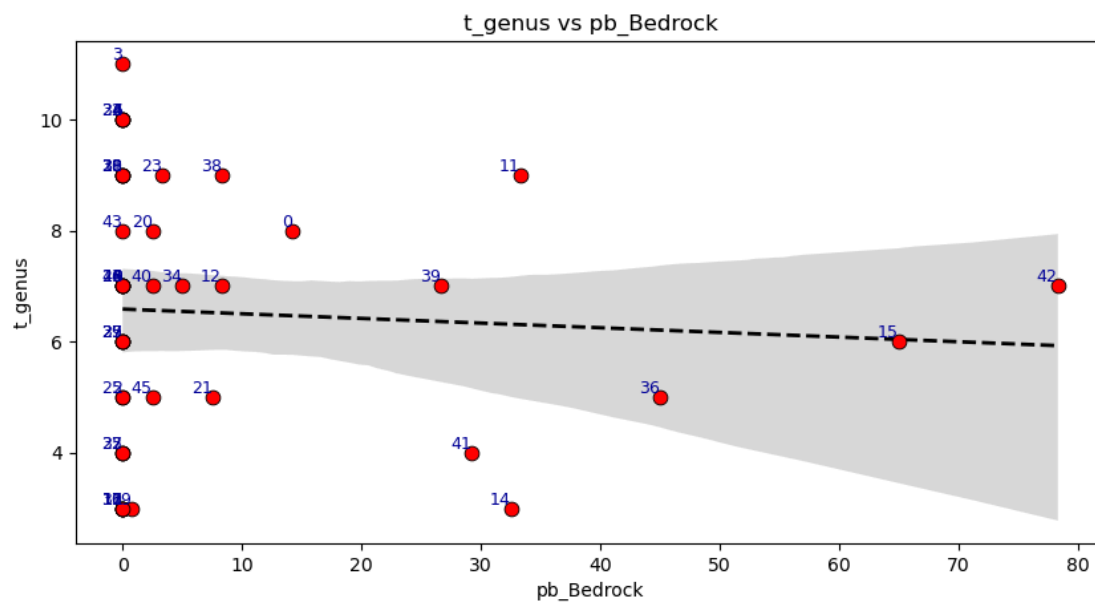
```
In [31]: scatter_trendLine_plotter(s_ed['pb_Bedrock'], s_egcd['t_population'], save=False)
scatter_trendLine_plotter(s_ed['pb_Bedrock'], s_egcd['t_genus'], save=False)
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

kbrashears_fishData_report_02042026

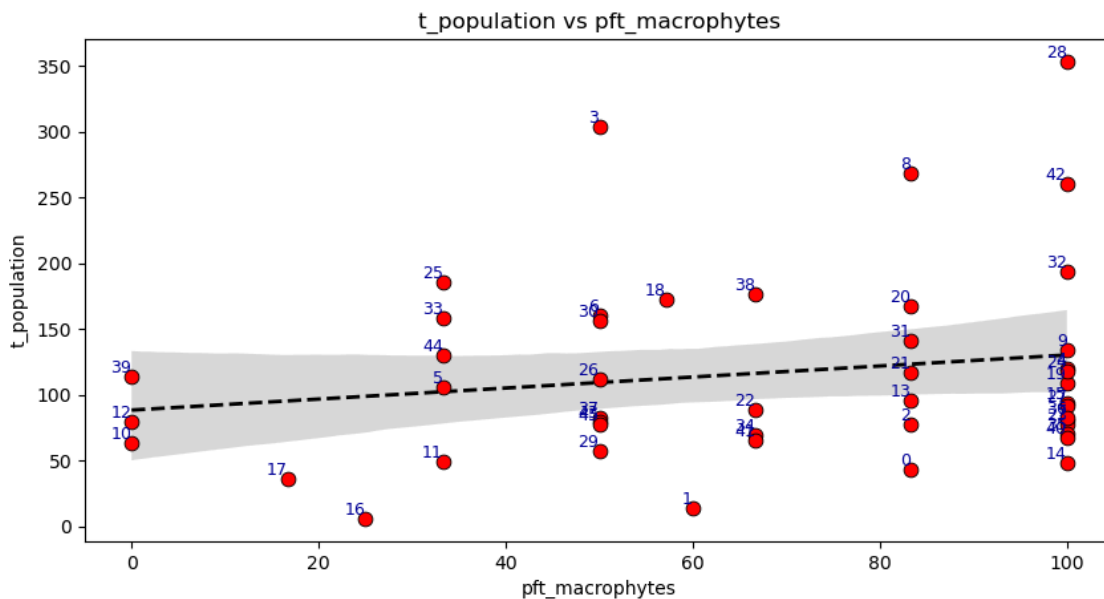
Korbin Brashears
02/15/2026

seth_envData_2025.csv

pft_macrophytes Scatter Plot

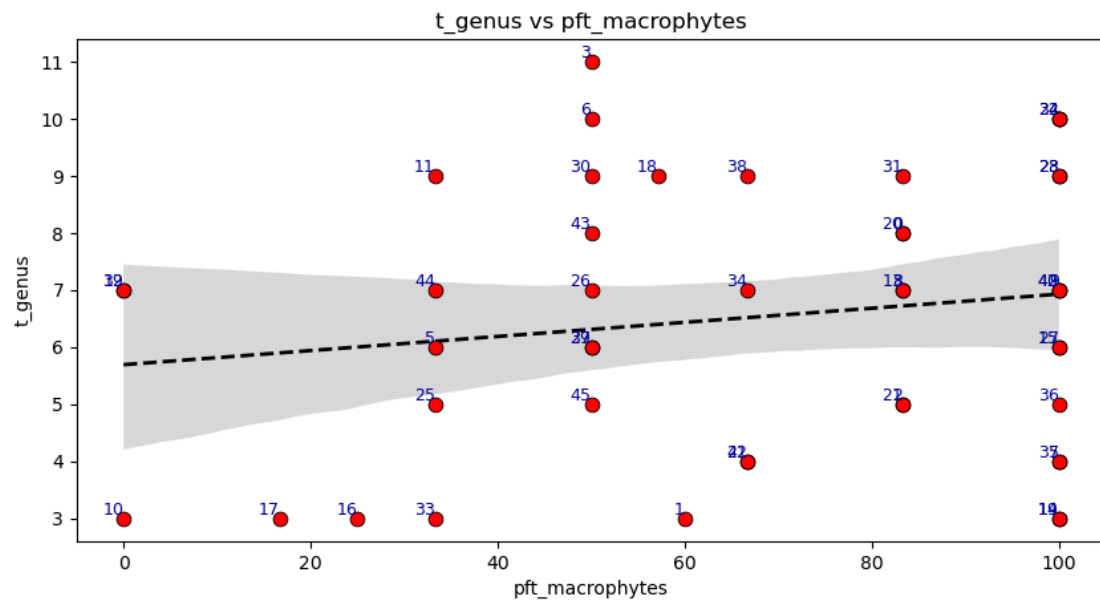
```
In [32]: scatter_trendLine_plotter(s_ed['pft_macrophytes'], s_egcd['t_population'], save=False)
scatter_trendLine_plotter(s_ed['pft_macrophytes'], s_egcd['t_genus'], save=False)
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

Clustering Methods

Here I will using various methods of clustering, as well as with various features and parameters till I find consistent clusters.

Clustering Sites by envAtt

```
In [ ]: cf_c = s_ed.columns[4:] # selecti
X = s_ed[cf_c].copy() # creatin
le = LabelEncoder() # creatin
labels = s_ed['State'].astype(str) # selecti
y = le.fit_transform(labels) # fitting
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scalar = StandardScaler() # creatin
Xtr = scalar.fit_transform(X_train) # fitting
Xte = scalar.transform(X_test) # fitting
#-----
# U-MAP Clustering
#-----
```

```

umap_model = umap.UMAP(
    n_components=2, metric='correlation', random_state=30,
    n_neighbors=7, n_epochs=700, learning_rate=0.01, min_dist=0.1, n_jobs=2
)

Xtr_umap = umap_model.fit_transform(Xtr)           # fitting
Xte_umap = umap_model.transform(Xte)              # transfo

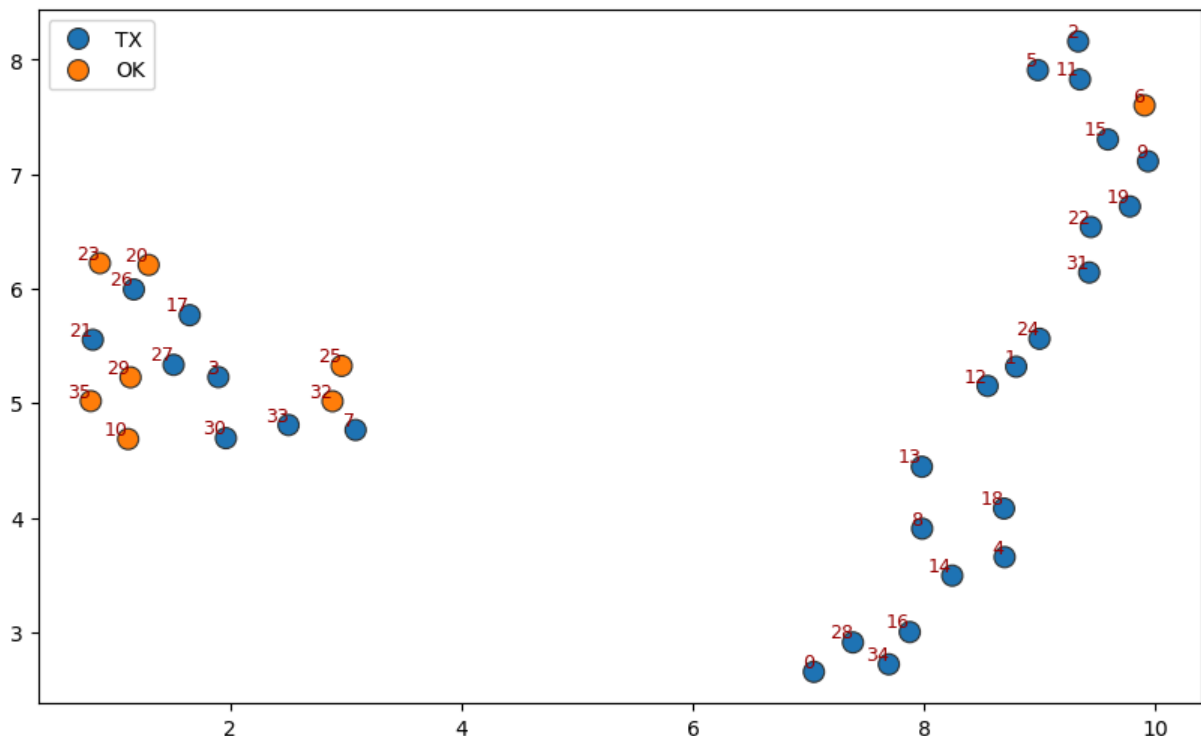
plt.figure(figsize=(10, 6))                        # creatin
sns.scatterplot(x=Xtr_umap[:, 0], y=Xtr_umap[:, 1], hue=le.inverse_transform(y_train))

# Adding Point Index Labels
for i in range(len(Xtr_umap[:, 0])):
    plt.text(Xtr_umap[i, 0], Xtr_umap[i, 1], str(i), fontsize=9, ha='right', va='bottom')

plt.show()

```

c:\Users\Korbin\anaconda3\Lib\site-packages\umap\umap.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(



KNN Classifier

A Focus on Orange Belly Darters

Seth has stated that *Etheostoma Radiosum*, ore the Orange Belly Darter, is of conservation interest, so I will be taking a moment to create some visualizations specifically on them.

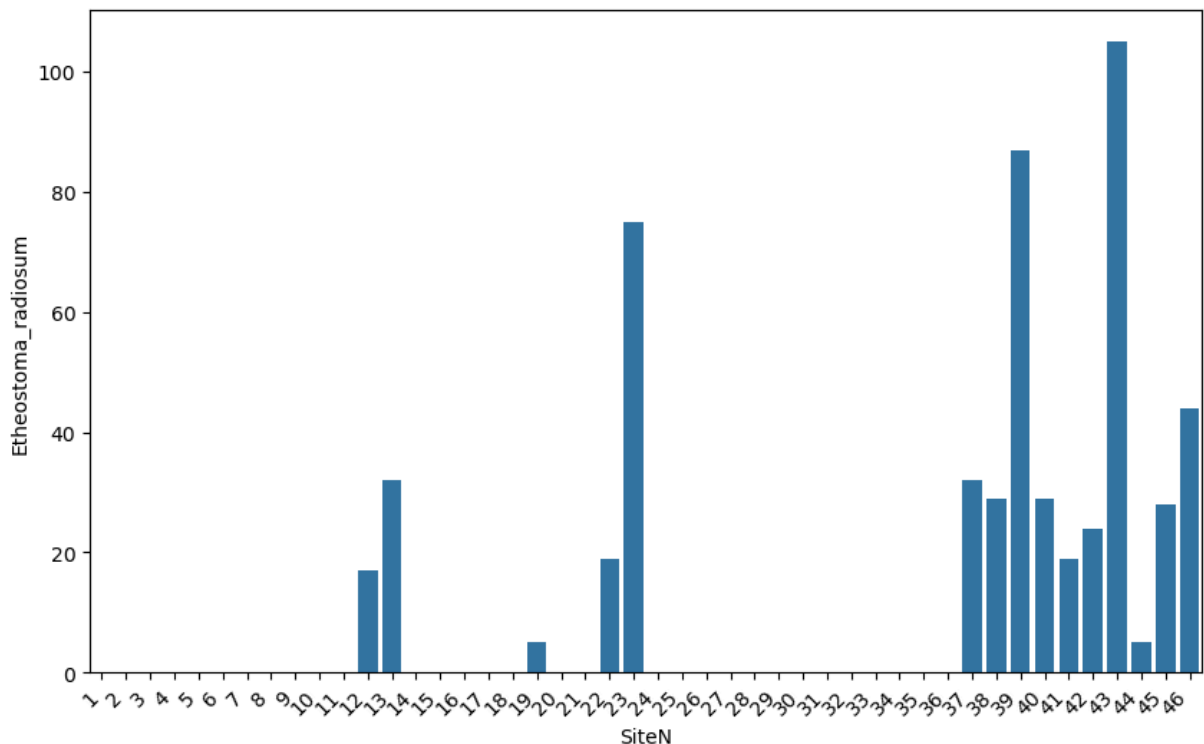
Etheostoma Radium Distribution

An interesting observation from this plot shows that the orange belly darter is primarily present in OK, with almost no sites in the TX region has any.

I will try to run **site_attributes_bar_plotter()** on the some of the significant TX sites containing the Orange Belly Darter, as well as some of the sites in OK and compare features.

POTENTIAL FUTURE SIGNIFICANCE Look at the day time series metrics to see any significant changes in the TX sites neat to the OK border

```
In [33]: plt.figure(figsize=(10, 6))                                     # creatin
sns.barplot(x=s_fcd['SiteN'], y=s_fcd['Etheostoma_radium'])           # creatin
plt.xticks(rotation=45, ha='right')                                    # rotating
plt.show()
```



Sites Attributes with High Orange Belly Darter Count

Taking a closer look into sites that have a high count of Etheostoma Radium using **site_attributes_bar_plotter**.

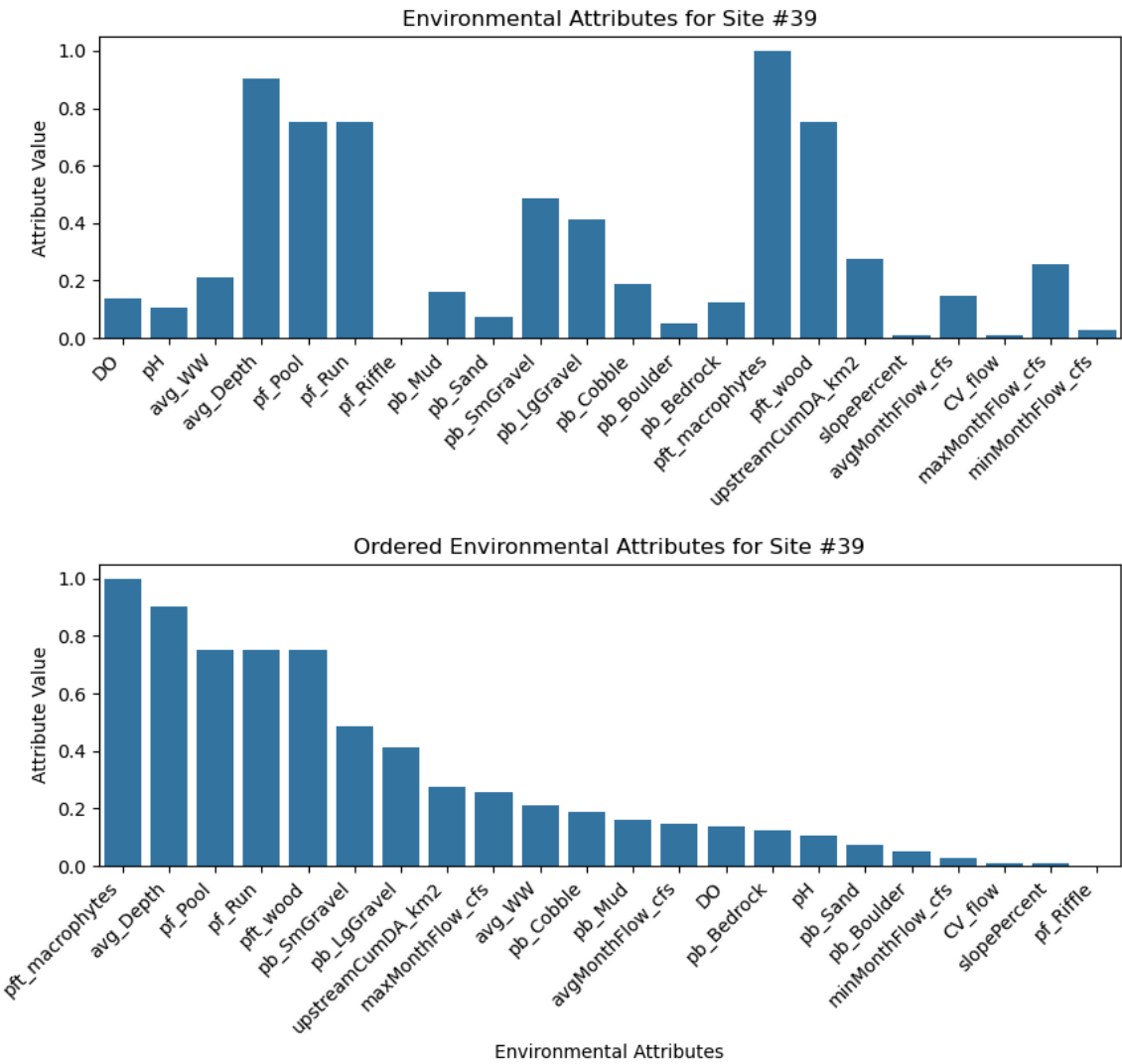
Site 39 is the site with the most Etheostoma Radium over all. Site 23 seems to be an outlier with a high count but located in Texas.

```
In [34]: # Site 39
site_attributes_bar_plotter(siteN=39, save=False) # calling the function to c

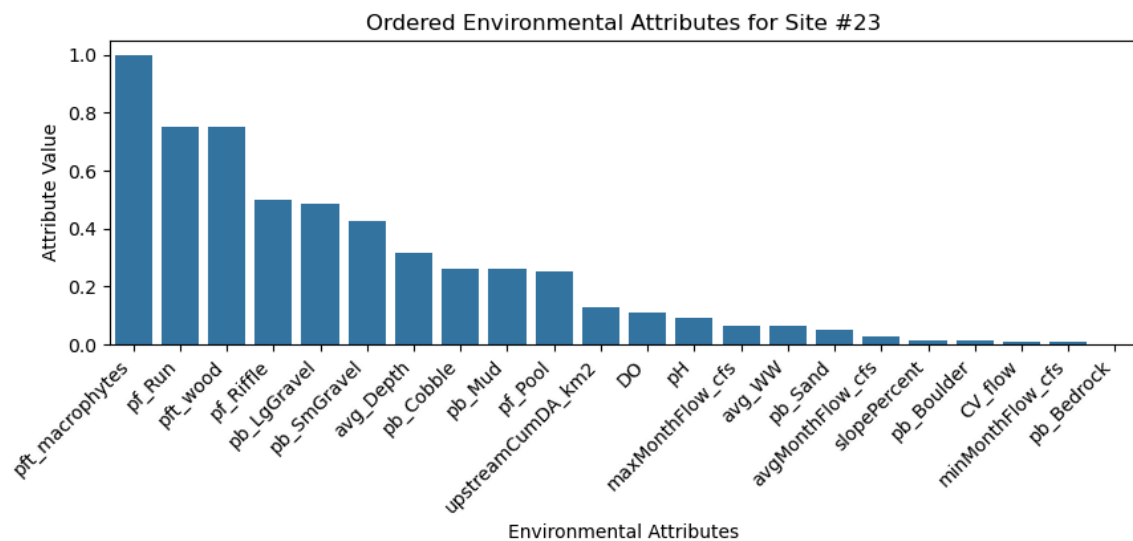
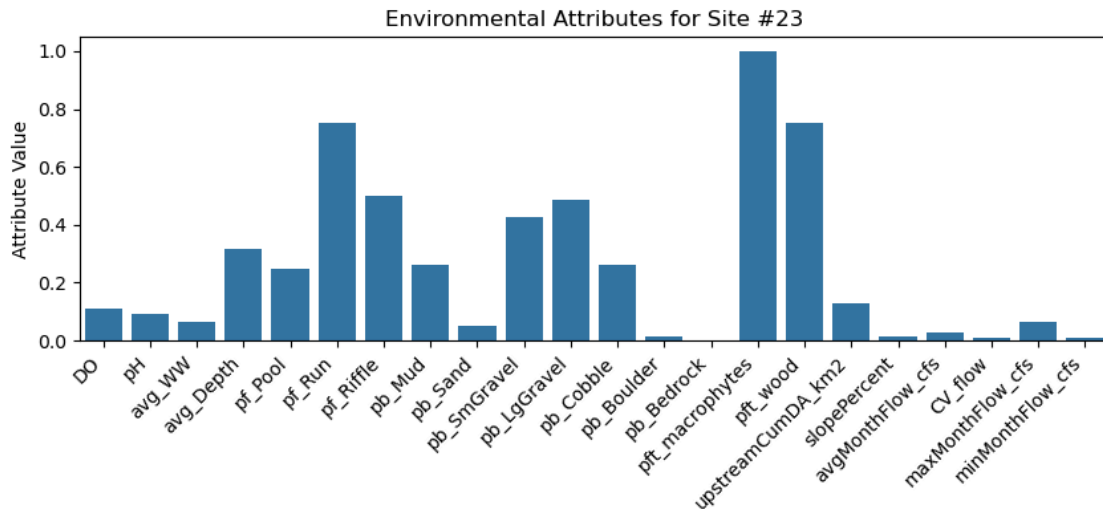
# Site 23
site_attributes_bar_plotter(siteN=23, save=False) # calling the function to c
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026



seth_envData_2025.csv



seth_envData_2025.csv

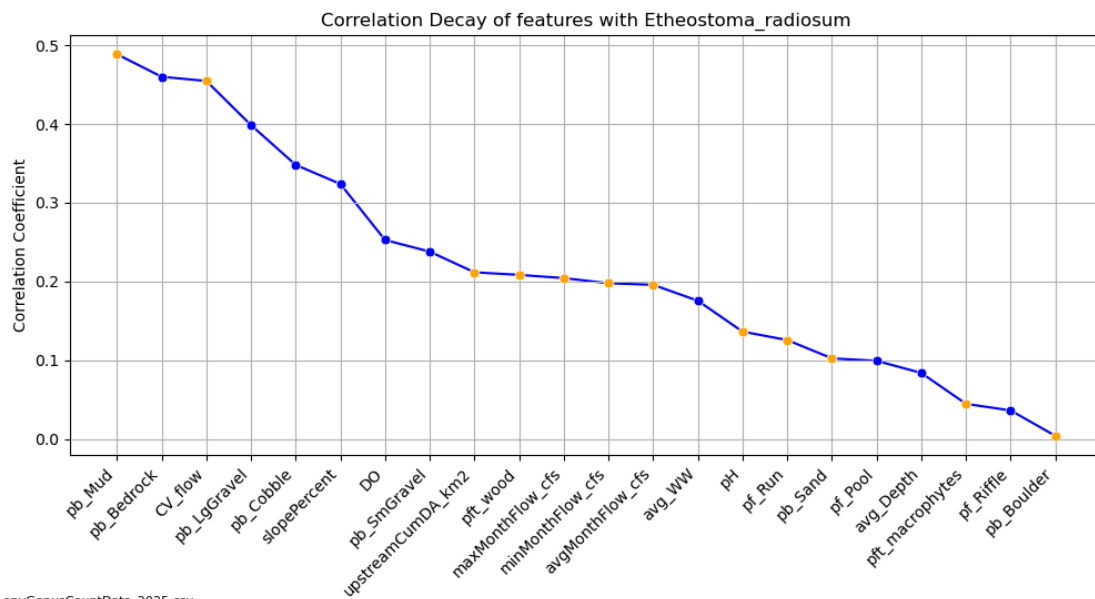
Correlation Decay Plot With Site Attributes

Correlation decay plot shows the most correlated attributes, x, to a feature, y, from highest to lowest. The correlation is in absolute value to make it easier to see the correlation, but the negatives are shown in orange to see inverse correlations.

```
In [35]: # Selecting Attribute Columns
att_cols = s_ed.columns[4:]

# Correlation Decay Plot
corr_decay_plotter(attribute=s_fcd['Etheostoma radiosum'], features=s_ed[att_cols],
```

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

Specific Attribute Trend Lines

Plotting a scatter plot and trend line from *Etheostoma Ramosum* vs Attributes of interest from the last plot.

```
In [36]: # pb_Mud
scatter_trendLine_plotter(s_ed['pb_Mud'], s_fcd['Etheostoma_ramosum'], save=False)

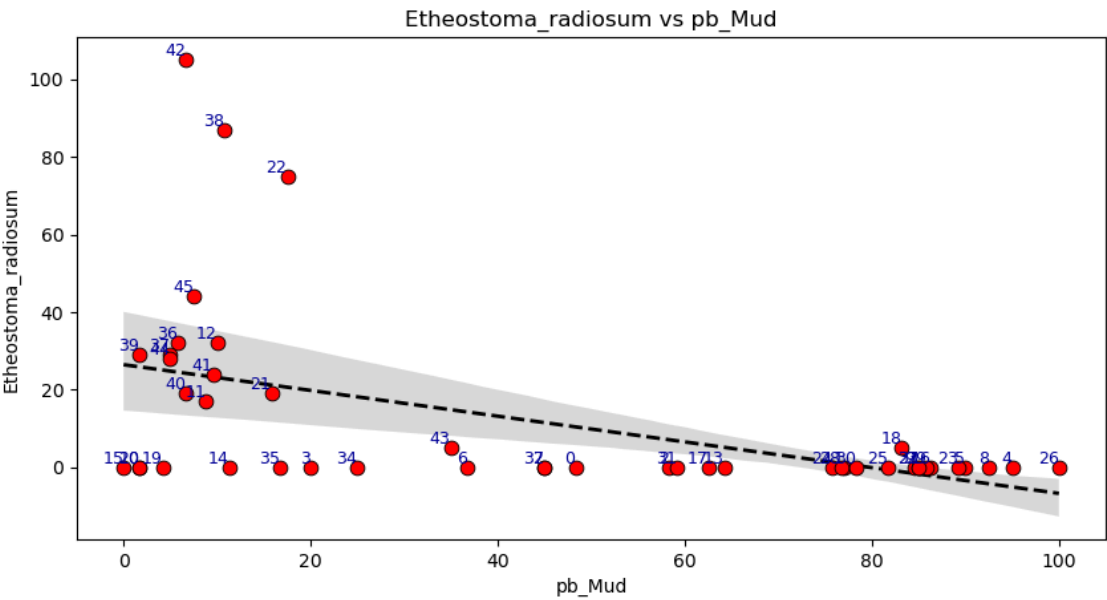
# CV_flow
scatter_trendLine_plotter(s_ed['CV_flow'], s_fcd['Etheostoma_ramosum'], save=False)

# slopePercent
scatter_trendLine_plotter(s_ed['slopePercent'], s_fcd['Etheostoma_ramosum'], save=False)

# pb_Bedrock
scatter_trendLine_plotter(s_ed['pb_Bedrock'], s_fcd['Etheostoma_ramosum'], save=False)
```

kbrashears_fishData_report_02042026

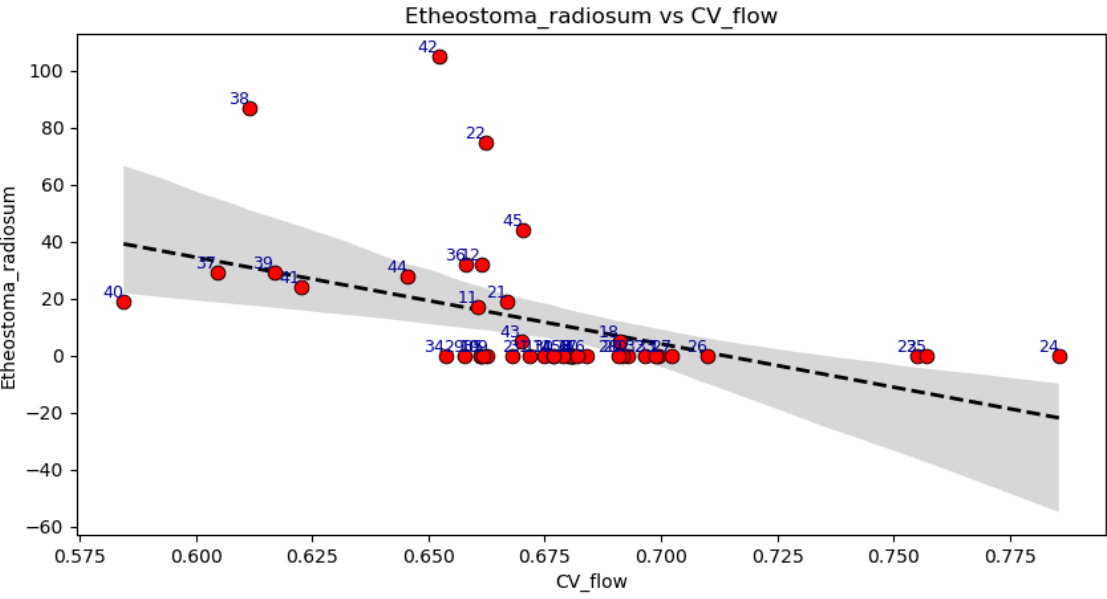
Korbin Brashears
02/15/2026



seth_envData_2025.csv

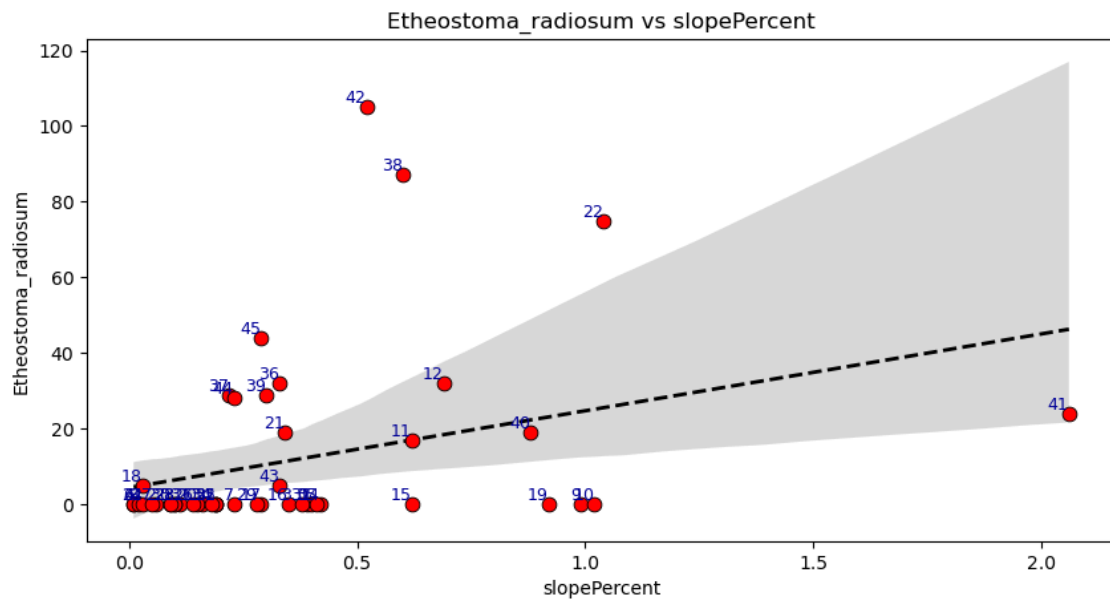
kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026



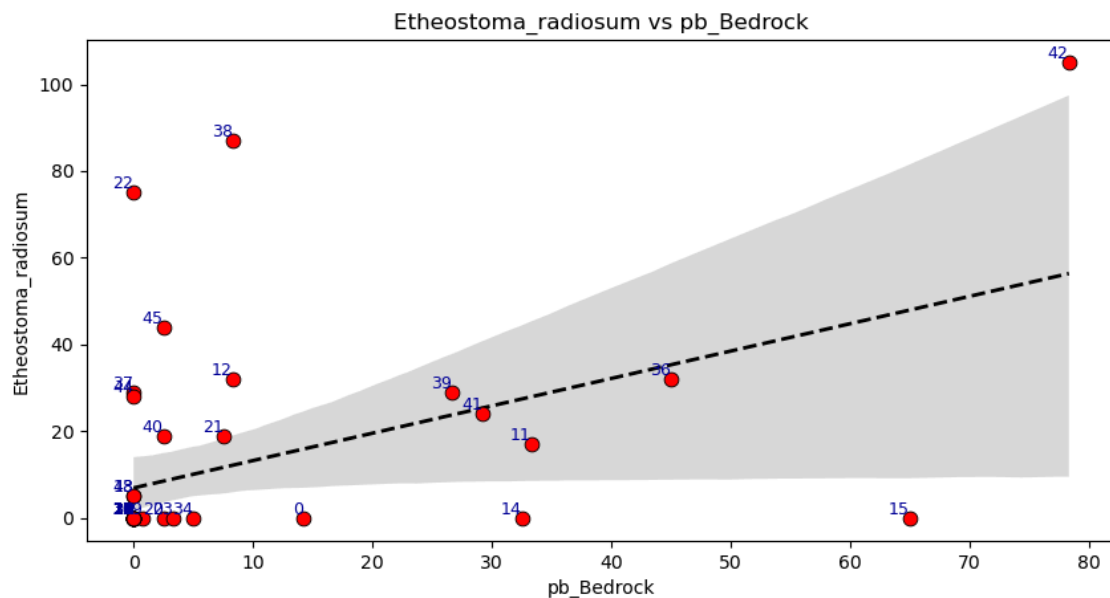
seth_envData_2025.csv

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

kbrashears_fishData_report_02042026

Korbin Brashears
02/15/2026

seth_envData_2025.csv

U-MAP Clustering

```
In [ ]: cf_c = ['pb_Bedrock', 'CV_flow', 'slopePercent'] # selecti
X = s_ed[cf_c].copy() # creatin
le = LabelEncoder() # creatin
```

```

labels = s_erd['State'].astype(str)                                # selecti
y = le.fit_transform(labels)                                       # fitting

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scalar = StandardScaler()                                         # creatin
Xtr = scalar.fit_transform(X_train)                                # fitting
Xte = scalar.transform(X_test)                                     # fitting

#-----
# U-MAP Clustering
#-----

umap_model = umap.UMAP(
    n_components=2, metric='correlation', random_state=30,
    n_neighbors=7, n_epochs=700, learning_rate=0.01, min_dist=0.1, n_jobs=2
)

Xtr_umap = umap_model.fit_transform(Xtr)                           # fitting
Xte_umap = umap_model.transform(Xte)                               # transfo

plt.figure(figsize=(10, 6))                                       # creatin
sns.scatterplot(x=Xtr_umap[:, 0], y=Xtr_umap[:, 1], hue=le.inverse_transform(y_train))

# Adding Point Index Labels
for i in range(len(Xtr_umap[:, 0])):
    plt.text(Xtr_umap[i, 0], Xtr_umap[i, 1], str(i), fontsize=9, ha='right', va='bottom')

plt.show()

# ===== Changing Labels =====

labels = []
for i in s_fcd['Etheostoma radiosum']:
    if i > 0:
        labels.append('Present')
    else:
        labels.append('Absent')
y = le.fit_transform(labels)                                       # fitting

plt.figure(figsize=(10, 6))                                       # creatin
sns.scatterplot(x=Xtr_umap[:, 0], y=Xtr_umap[:, 1], hue=le.inverse_transform(y_train))

# Adding Point Index Labels
for i in range(len(Xtr_umap[:, 0])):
    plt.text(Xtr_umap[i, 0], Xtr_umap[i, 1], str(i), fontsize=9, ha='right', va='bottom')

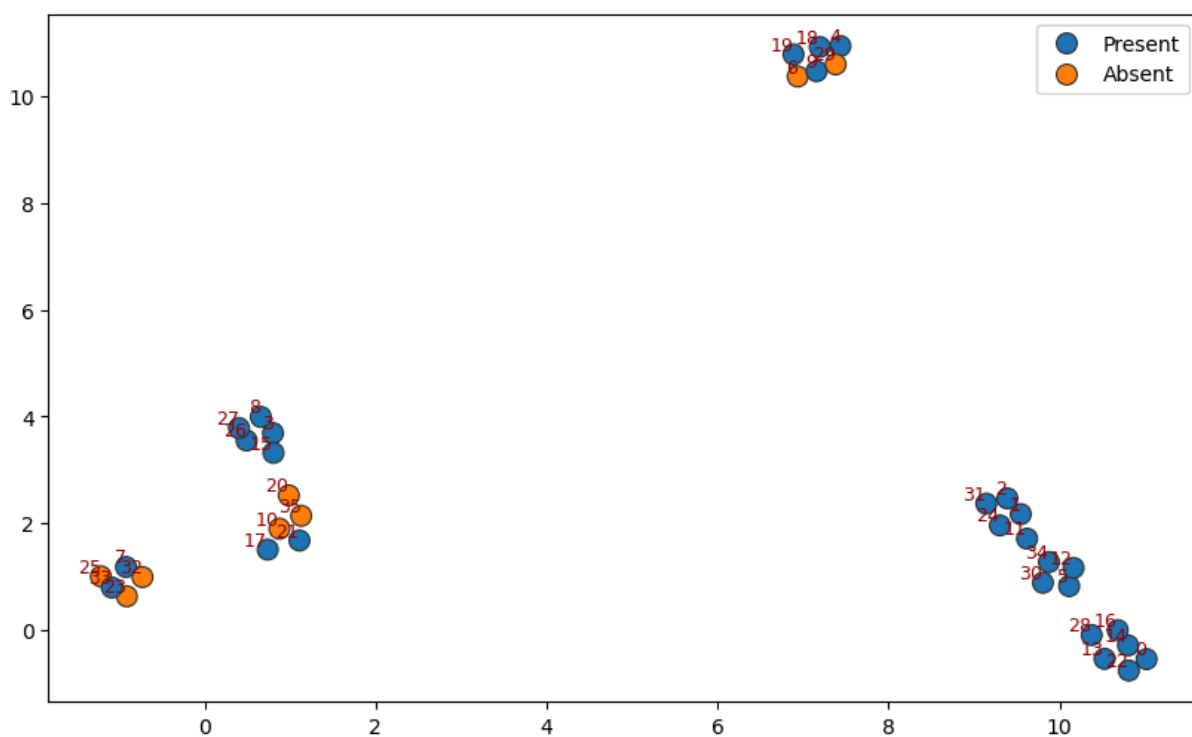
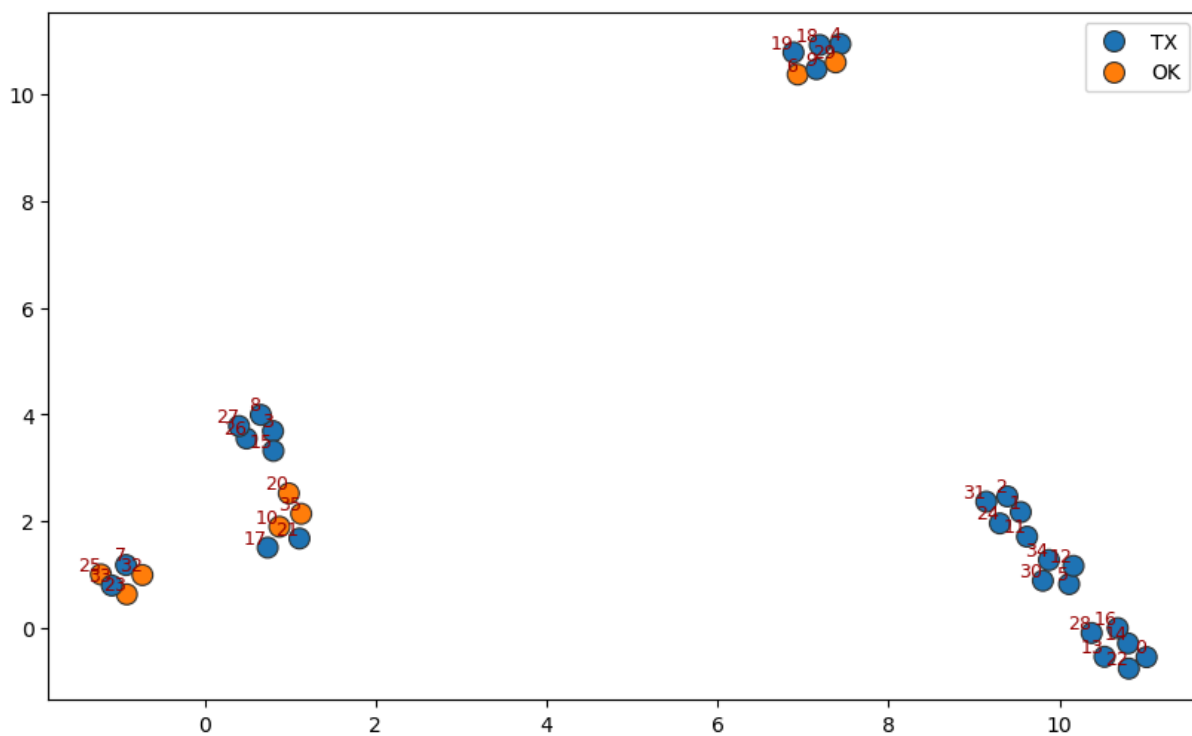
plt.show()

```

```

c:\Users\Korbin\anaconda3\Lib\site-packages\umap\umap_.py:1952: UserWarning: n_jobs
value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(

```



KNN Classifier

This cell creates a KNN classifier to predict if a site does or does not have Orange Belly Darters.

```
In [52]: s_ed['has_Etheostoma_radiosum'] = s_fcd['Etheostoma_radiosum'] > 0 # creating
         cf_c = ['pb_Bedrock', 'CV_flow', 'slopePercent'] # selecti
```

```

X = s_ed[cf_c].copy() # creatin

y = s_ed['has_Etheostoma_radiosum'].astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scalar = StandardScaler() # creatin
Xtr = scalar.fit_transform(X_train) # fitting
Xte = scalar.transform(X_test) # fitting

#-----
# KNN Clustering
#-----

knn_model = KNeighborsClassifier(n_neighbors=3) # creati
knn_model.fit(Xtr, y_train) # fittin
y_pred = knn_model.predict(Xte) # using
print(classification_report(y_test, y_pred, target_names=['False', 'True'])) #

# Creating Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred False", "Pred True"],
            yticklabels=["Actual False", "Actual True"])
plt.title("Confusion Matrix - KNN (k=3)")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.show()

```

	precision	recall	f1-score	support
False	0.86	0.86	0.86	7
True	0.67	0.67	0.67	3
accuracy			0.80	10
macro avg	0.76	0.76	0.76	10
weighted avg	0.80	0.80	0.80	10

